

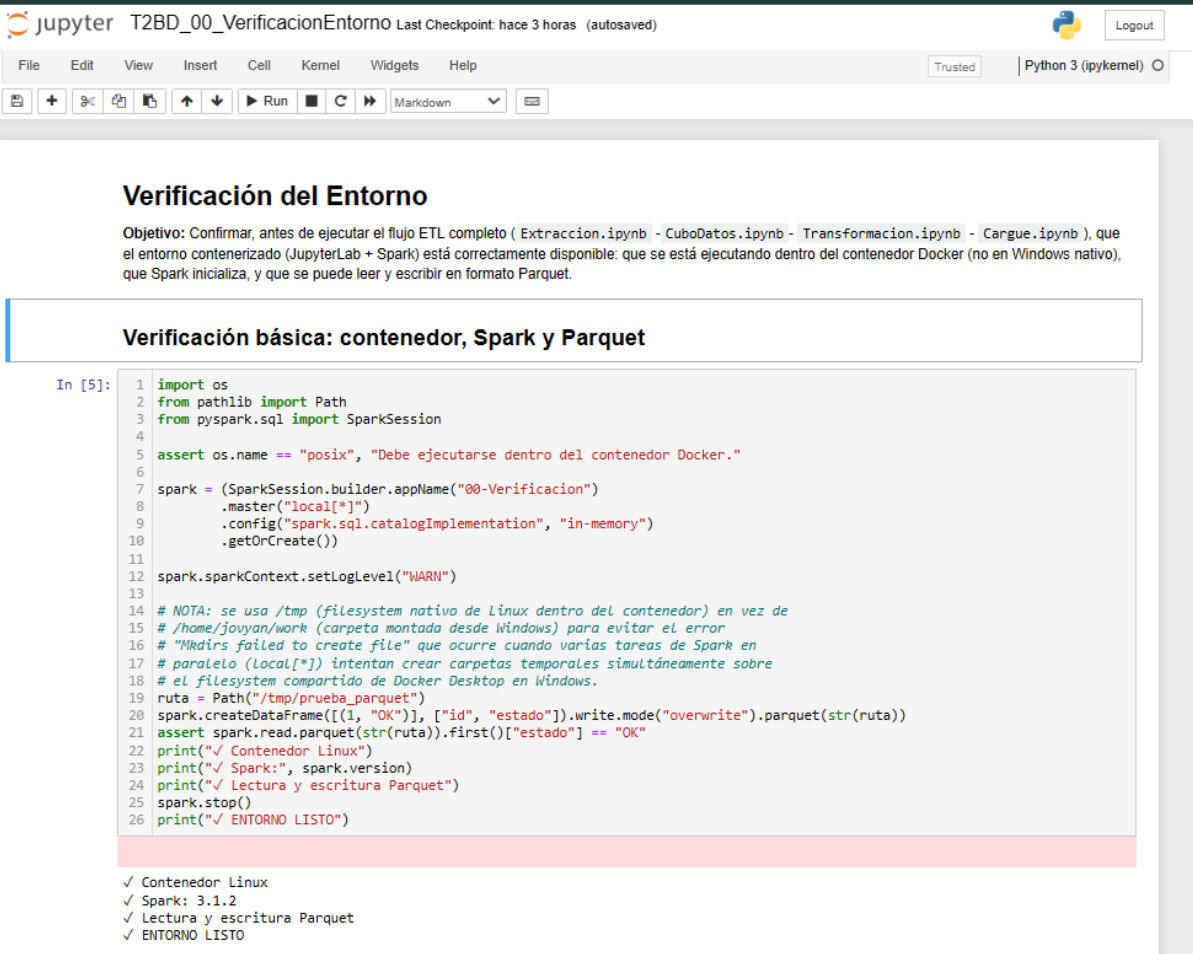
Taller ETL – Cubo SECOP

Autor: Jurani Zabala Hernandez

El objetivo de este documento es el registro de los resultados obtenidos en Jupyterlab en los 4 notebooks solicitados en el ejercicio + 2 adicionales:

T2BD_00_VerificacionEntorno.ipynb y T2BD_ConsultasSparkSQL.ipynb, correspondientes a la implementación de ETL en todo el taller.

Se prepara inicialmente un notebook de verificación del entorno con el fin de Confirmar, antes de ejecutar el flujo ETL completo (Extraccion.ipynb -CuboDatos.ipynb- Transformacion.ipynb - Cargue.ipynb), que el entorno contenerizado (JupyterLab + Spark) está correctamente disponible: que se está ejecutando dentro del contenedor Docker (no en Windows nativo), que Spark inicializa, y que se puede leer y escribir en formato Parquet.



Verificación del Entorno

Objetivo: Confirmar, antes de ejecutar el flujo ETL completo (Extraccion.ipynb - CuboDatos.ipynb - Transformacion.ipynb - Cargue.ipynb), que el entorno contenerizado (JupyterLab + Spark) está correctamente disponible: que se está ejecutando dentro del contenedor Docker (no en Windows nativo), que Spark inicializa, y que se puede leer y escribir en formato Parquet.

Verificación básica: contenedor, Spark y Parquet

```
In [5]: 1 import os
2 from pathlib import Path
3 from pyspark.sql import SparkSession
4
5 assert os.name == "posix", "Debe ejecutarse dentro del contenedor Docker."
6
7 spark = (SparkSession.builder.appName("00-Verificacion")
8         .master("local[*]")
9         .config("spark.sql.catalogImplementation", "in-memory")
10        .getOrCreate())
11
12 spark.sparkContext.setLogLevel("WARN")
13
14 # NOTA: se usa /tmp (filesystem nativo de Linux dentro del contenedor) en vez de
15 # /home/jovyan/work (carpeta montada desde Windows) para evitar el error
16 # "Mkdirs failed to create file" que ocurre cuando varias tareas de Spark en
17 # paralelo (Local[*]) intentan crear carpetas temporales simultáneamente sobre
18 # el filesystem compartido de Docker Desktop en Windows.
19 ruta = Path("/tmp/prueba_parquet")
20 spark.createDataFrame([(1, "OK")], ["id", "estado"]).write.mode("overwrite").parquet(str(ruta))
21 assert spark.read.parquet(str(ruta)).first()["estado"] == "OK"
22 print("✓ Contenedor Linux")
23 print("✓ Spark:", spark.version)
24 print("✓ Lectura y escritura Parquet")
25 spark.stop()
26 print("✓ ENTORNO LISTO")
```

✓ Contenedor Linux
✓ Spark: 3.1.2
✓ Lectura y escritura Parquet
✓ ENTORNO LISTO

Extracción de datos SECOP II (Bronce)

Taller ETL – Cubo SECOP Autor: Jurani Zabala Hernandez **Objetivo:** Extraer la información de contratación pública desde la API oficial del SECOP II (SODA v3 / datos.gov.co) y depositarla, sin transformar, en el área **bronce** del *data lake* (HDFS), como insumo para los siguientes notebooks del taller.

Alcance notebook:

1. Conexión a Spark con soporte HDFS/Hive de acuerdo a requerimientos solicitados
2. Extracción paginada y robusta desde la API pública del SECOP II, con manejo de errores y reintentos.
3. Persistencia cruda (sin transformar) en el data lake, área **bronce**, **página por página** (no se acumula el dataset completo en memoria).
4. Validación básica de la extracción (conteo de registros, esquema).

1. Configuración del entorno y sesión de Spark

Se inicializa Spark apuntando al namenode definido en el docker-compose.yml del entorno containerizado provisto en el taller, con soporte Hive habilitado (necesario para los pasos posteriores).

```
.j: 1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import current_timestamp, lit
3
4 spark = (
5     SparkSession.builder
6     .appName("SECOP_Extraccion_Bronce")
7     .master("local[*]")
8     .config("spark.driver.memory", "4g")
9     .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000")
10    .config("spark.sql.catalogImplementation", "hive")
11    .config("spark.sql.execution.arrow.pyspark.enabled", "true") # conversión pandas->Spark mucho más rápida
12    .enableHiveSupport()
13    .getOrCreate()
14 )
15
16 spark.sparkContext.setLogLevel("WARN")
17 print("✅ Sesión Spark inicializada:", spark.version)
```

✅ Sesión Spark inicializada: 3.1.2

2. Extracción desde la API SECOP II

Se utiliza la API SODA v3 del portal de datos abiertos de Colombia, tal como está enlazada en la guía del taller:

1. Dataset: SECOP II - Contratos Electrónicos (ibjy-vk9h) — dataset completo: ~5.98 millones de filas x 85 columnas.
2. Endpoint: <https://www.datos.gov.co/api/v3/views/ibjy-vk9h/query.json>
3. Límite real de la API: Socrata (SODA) impone un máximo de 50,000 registros por página, sin importar qué valor de pageSize se solicite.
4. De acuerdo a lo anterior y para efectos de este taller se define PAGE_SIZE = 2000 y TOTAL_LIMIT = 20000 para que ejecute adecuadamente todos los pasos y limitar el volumen por razones de tiempo/recursos para que la extracción esté bien implementada (paginación, manejo de errores, cobertura de fragmentos)
5. Diseño para no saturar la memoria del driver: con ~5.98M de filas y 85 columnas, acumular todo el dataset en una sola lista de Python antes de escribirlo puede agotar la memoria del contenedor de Jupyter (configurado con 4 GB para el driver de Spark). Por eso este notebook escribe cada página a HDFS inmediatamente después de descargarla, en vez de acumular todo y escribir al final.

Buenas prácticas aplicadas:

1. Paginación completa mediante pageNumber / pageSize (máximo permitido por la API) hasta agotar los registros disponibles o el tope configurado en TOTAL_LIMIT.
2. Control robusto de errores: reintentos con backoff exponencial ante fallos de red o códigos HTTP distintos de 200.
3. Escritura incremental (página por página) para soportar volúmenes grandes sin agotar memoria.
4. Registro de metadatos de auditoría (fecha de extracción, página, tamaño de lote).

```
Descargando página 1 (tamaño 5000) ...
Descargando página 2 (tamaño 5000) ...
Descargando página 3 (tamaño 5000) ...
Descargando página 4 (tamaño 5000) ...
```

✅ Total de registros descargados: 20000

3. Extracción y persistencia incremental (página por página)

En vez de acumular todas las páginas en memoria y escribir al final, cada página se convierte a DataFrame de Spark y se agrega (`append`) inmediatamente al área bronce. Esto permite procesar el dataset completo (millones de filas) sin necesidad de que quepa todo junto en la memoria del driver.

- Persistencia en el área Bronce del Data Lake Los datos se guardan tal como llegan de la fuente (sin normalizar ni tipar), en formato Parquet, particionados por fecha de ingesta. Esto sigue el patrón medallion (bronce - plata - oro) solicitado para este taller:

Bronce (esta etapa): datos crudos, fieles a la fuente. Plata: se genera en Transformacion.ipynb. Oro: se genera en Cargue.ipynb, ya cargado en el cubo Hive.

```
Descargando página 1 (tamaño 2000) ...
```

```
26/08/24 00:05:46 WARN package: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
```

```
-> 2000 registros escritos en bronce. Acumulado: 2000
```

```
Descargando página 2 (tamaño 2000) ...
```

```
-> 2000 registros escritos en bronce. Acumulado: 4000
```

```
Descargando página 3 (tamaño 2000) ...
```

```
-> 2000 registros escritos en bronce. Acumulado: 6000
```

```
Descargando página 4 (tamaño 2000) ...
```

```
-> 2000 registros escritos en bronce. Acumulado: 8000
```

```
Descargando página 5 (tamaño 2000) ...
```

```
-> 2000 registros escritos en bronce. Acumulado: 10000
```

```
Descargando página 6 (tamaño 2000) ...
```

```
-> 2000 registros escritos en bronce. Acumulado: 12000
```

```
Descargando página 7 (tamaño 2000) ...
```

```
-> 2000 registros escritos en bronce. Acumulado: 14000
```

```
Descargando página 8 (tamaño 2000) ...
```

```
-> 2000 registros escritos en bronce. Acumulado: 16000
```

```
Descargando página 9 (tamaño 2000) ...
```

```
-> 2000 registros escritos en bronce. Acumulado: 18000
```

```
Descargando página 10 (tamaño 2000) ...
```

```
-> 2000 registros escritos en bronce. Acumulado: 20000
```

✅ Extracción finalizada. Total de registros persistidos en bronce: 20000

3. Validación rápida de la extracción

Se revisa la forma de los datos crudos antes de persistirlos: número de columnas, muestra de filas y nulos evidentes.

```
In [4]: 1 df_pandas = pd.DataFrame(registros)
2 print(f"Filas: {len(df_pandas)} | Columnas: {len(df_pandas.columns)}")
3 df_pandas.head()
```

Filas: 20000 | Columnas: 85

```
Out[4]:
```

	nombre_entidad	nit_entidad	departamento	ciudad	localizaci_n	orden	sector	rama	entidad_centralizada	proceso_de_compra	...	nombre_c
0	ESE HOSPITAL DEPARTAMENTAL UNIVERSITARIO DEL Q...	800000118	Quindío	Armenia	Colombia, Quindío, Armenia	Nacional	Salud y Protección Social	Corporación Autónoma	Centralizada	CO1.BDOS.3452554	...	
1	MUNICIPIO DE SOLEDAD	890106291	Atlántico	Soledad	Colombia, Atlántico, Soledad	Territorial	Servicio Público	Ejecutivo	Centralizada	CO1.BDOS.9960799	...	EDISON
2	Ministerio de las Culturas, las Artes y los Sa...	830003438	Distrito Capital de Bogotá	Bogotá	Colombia, Bogotá, Bogotá	Nacional	Cultura	Ejecutivo	Centralizada	CO1.BDOS.8509712	...	
3	HOSPITAL DEPARTAMENTAL SAN JUAN DE DIOS DE PUE...	842000004	Vichada	Puerto Carreño	Colombia, Vichada, Puerto Carreño	Territorial	Salud y Protección Social	Corporación Autónoma	Centralizada	CO1.BDOS.9690037	...	
4	CONTRALORIA DEPARTAMENTAL DE NARIÑO	800157830	Nariño	No Definido	Colombia, Nariño, No Definido	Territorial	No aplica/No pertenece	Corporación Autónoma	Descentralizada	CO1.BDOS.5042930	...	

5 rows x 85 columns

```
In [5]: 1 # Se agrega metadata de auditoría de la extracción antes de convertir a Spark
2 df_pandas["_fecha_extraccion"] = fecha_extraccion
3 df_pandas["_fuente"] = "SECOPII_API_datos.gov.co"
4
5 df_raw = spark.createDataFrame(df_pandas.astype(str)) # se castea a string para evitar inferencias erróneas de tipo en el d
6 df_raw.printSchema()
7 df_raw.show(5, truncate=True)
```

```
-- modalidad_de_contratacion: string (nullable = true)
-- justificacion_modalidad_de: string (nullable = true)
-- fecha_de_firma: string (nullable = true)
-- fecha_de_inicio_del_contrato: string (nullable = true)
-- fecha_de_fin_del_contrato: string (nullable = true)
-- condiciones_de_entrega: string (nullable = true)
-- tipodocproveedor: string (nullable = true)
-- documento_proveedor: string (nullable = true)
-- proveedor_adjudicado: string (nullable = true)
-- es_grupo: string (nullable = true)
-- es_pyme: string (nullable = true)
-- habilita_pago_adelantado: string (nullable = true)
-- liquidaci_n: string (nullable = true)
-- obligaci_n_ambiental: string (nullable = true)
-- obligaciones_postconsumo: string (nullable = true)
-- reversion: string (nullable = true)
-- origen_de_los_recursos: string (nullable = true)
-- destino_gasto: string (nullable = true)
-- valor_del_contrato: string (nullable = true)
-- valor_de_pago_adelantado: string (nullable = true)
```

5. Verificación de la persistencia

```
In [6]: 1 df_check = spark.read.parquet(BRONZE_PATH)
2 print(f"Registros totales en bronce (todas las ingestas): {df_check.count()}")
3 df_check.select("id_contrato", "nombre_entidad", "fecha_ingesta").show(5, truncate=False)
```

Registros totales en bronce (todas las ingestas): 20000

id_contrato	nombre_entidad	fecha_ingesta
CO1.PCCNTR.4675714	DEPARTAMENTO DE POLICIA GUAINIA - DEGUN	2026-08-24
CO1.PCCNTR.9795526	DEFENSORÍA DEL PUEBLO	2026-08-24
CO1.PCCNTR.4384661	INSTITUCIÓN UNIVERSITARIA ITM	2026-08-24
CO1.PCCNTR.6694657	ALCALDIA LOCAL DE SAN CRISTOBAL	2026-08-24
CO1.PCCNTR.8560944	DEPARTAMENTO DE SANTANDER	2026-08-24

only showing top 5 rows

Conclusiones

- Se implementó la extracción completa desde la API pública del SECOP II mediante paginación y con LIMITE TOTAL de registros
- Se incorporó control robusto de errores con reintentos y backoff exponencial ante fallos de red o HTTP.
- Los datos crudos quedaron persistidos en el área **bronce** del *data lake* en formato Parquet, particionados por fecha de ingesta, listos para ser consumidos por `Transformacion.ipynb`.

Implementación del Cubo de Datos SECOP (Hive)

Taller ETL – Cubo SECOP Autor: Jurani Zabala Hernandez

Objetivo: Traducir a un modelo tabular en Hive el diseño dimensional definido en `CuboContratosPostgreSQL.drawio` y dejarlo implementado en el *data lake*, listo para ser poblado por `Transformacion.ipynb` y `Cargue.ipynb`. Basado también en el esquema estrella con **7 dimensiones** — incluyendo `dim_tiempo` como **dimensión de rol**, referenciada 3 veces desde el hecho (fecha de firma, inicio y fin) — y **1 tabla de hechos** (`hecho_contratos`).

1. Sesión de Spark con soporte Hive

```
In [1]: 1 from pyspark.sql import SparkSession
2
3 spark = (
4     SparkSession.builder
5     .appName("SECOP_CuboDatos")
6     .master("local[*]")
7     .config("spark.driver.memory", "4g")
8     .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000")
9     .config("spark.sql.catalogImplementation", "hive")
10    .enableHiveSupport()
11    .getOrCreate()
12 )
13
14 spark.sparkContext.setLogLevel("WARN")
15 print("✅ Sesión Spark con soporte Hive inicializada:", spark.version)
```

```
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.apache.spark.unsafe.Platform (file:/usr/local/spark-3.1.2-bin-hadoop3.2/jars/spark-unsafe_2.12-3.1.2.jar) to constructor java.nio.DirectByteBuffer(long,int)
WARNING: Please consider reporting this to the maintainers of org.apache.spark.unsafe.Platform
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
26/08/24 01:43:08 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
```

✅ Sesión Spark con soporte Hive inicializada: 3.1.2

2. Creación del esquema (base de datos) del cubo

```
In [2]: 1 DB_NAME = "secop_dw"
2
3 spark.sql(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}")
4 spark.catalog.setCurrentDatabase(DB_NAME)
5 print(f"✅ Base de datos '{DB_NAME}' creada/seleccionada.")
6 spark.sql("SHOW DATABASES").show(truncate=False)

26/08/24 01:43:15 WARN HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
26/08/24 01:43:15 WARN HiveConf: HiveConf of name hive.stats.retries.wait does not exist
26/08/24 01:43:24 WARN ObjectStore: Version information not found in metastore. hive.metastore.schema.verification is not enabled so recording the schema version 2.3.0
26/08/24 01:43:24 WARN ObjectStore: setMetaStoreSchemaVersion called but recording version is disabled: version = 2.3.0, comment = Set by MetaStore UNKNOWN@172.18.0.6
26/08/24 01:43:24 WARN ObjectStore: Failed to get database default, returning NoSuchObjectException
26/08/24 01:43:25 WARN ObjectStore: Failed to get database global_temp, returning NoSuchObjectException
26/08/24 01:43:25 WARN ObjectStore: Failed to get database secop_dw, returning NoSuchObjectException

✅ Base de datos 'secop_dw' creada/seleccionada.
+-----+
|namespace|
+-----+
|default   |
|secop_dw  |
+-----+
```

3. Creación de las 7 tablas de dimensión

```
In [3]: 1 spark.sql("""
2 CREATE TABLE IF NOT EXISTS {DB_NAME}.dim_entidad (
3     sk_entidad      STRING,
4     codigo_entidad  STRING,
5     nit_entidad     STRING,
6     nombre_entidad  STRING,
7     orden           STRING,
8     sector          STRING,
9     rama            STRING,
10    centralizada    STRING
11 )
12 STORED AS PARQUET
13 """)
14
15 spark.sql("""
16 CREATE TABLE IF NOT EXISTS {DB_NAME}.dim_proveedor (
17     sk_proveedor    STRING,
18     codigo_proveedor STRING,
19     tipo_documento  STRING,
20     documento_proveedor STRING,
21     nombre_proveedor STRING,
22     es_grupo        STRING,
23     es_pyme         STRING
24 )
25 STORED AS PARQUET
26 """)
27
28 spark.sql("""
29 CREATE TABLE IF NOT EXISTS {DB_NAME}.dim_tiempo (
30     sk_tiempo       STRING,
31     fecha           DATE,
32     anio            INT,
33     semestre        INT,
34     trimestre       INT,
35     mes             INT,
36     nombre_mes      STRING,
37     dia            INT
38 )
39 STORED AS PARQUET
```

```
84 print("✅ / tablas de dimension creadas.")
```

```
26/08/24 01:44:52 WARN SessionState: METASTORE_FILTER_HOOK will be ignored, since hive.security.authorization.manager is set to instance of HiveAuthorizerFactory.
26/08/24 01:44:52 WARN HiveConf: HiveConf of name hive.internal.ss.authz.settings.applied.marker does not exist
26/08/24 01:44:52 WARN HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
26/08/24 01:44:52 WARN HiveConf: HiveConf of name hive.stats.retries.wait does not exist
26/08/24 01:44:52 WARN HiveMetaStore: Location: file:/home/jovyan/work/spark-warehouse/secop_dw.db/dim_entidad specified for non-external table:dim_entidad
26/08/24 01:44:53 WARN HiveMetaStore: Location: file:/home/jovyan/work/spark-warehouse/secop_dw.db/dim_proveedor specified for non-external table:dim_proveedor
26/08/24 01:44:53 WARN HiveMetaStore: Location: file:/home/jovyan/work/spark-warehouse/secop_dw.db/dim_tiempo specified for non-external table:dim_tiempo
26/08/24 01:44:53 WARN HiveMetaStore: Location: file:/home/jovyan/work/spark-warehouse/secop_dw.db/dim_modalidad specified for non-external table:dim_modalidad
26/08/24 01:44:53 WARN HiveMetaStore: Location: file:/home/jovyan/work/spark-warehouse/secop_dw.db/dim_ubicacion specified for non-external table:dim_ubicacion
26/08/24 01:44:53 WARN HiveMetaStore: Location: file:/home/jovyan/work/spark-warehouse/secop_dw.db/dim_estado_contrato specified for non-external table:dim_estado_contrato
```

```
✅ 7 tablas de dimensión creadas.
```

```
26/08/24 01:44:54 WARN HiveMetaStore: Location: file:/home/jovyan/work/spark-warehouse/secop_dw.db/dim_categoria specified for non-external table:dim_categoria
```

4. Creación de la tabla de hechos

`dim_tiempo` se referencia 3 veces (`sk_tiempo_firma`, `sk_tiempo_inicio`, `sk_tiempo_fin`) — es una dimensión de rol. Se particiona por `anio_firma` (derivado de la fecha de firma) como optimización física, adicional al diagrama lógico pero estándar en implementaciones Hive de este tipo de cubo.

In [4]:

```
1 spark.sql("""
2 CREATE TABLE IF NOT EXISTS {DB_NAME}.hecho_contratos (
3     id_contrato          STRING,
4     sk_entidad            STRING,
5     sk_proveedor         STRING,
6     sk_tiempo_firma       STRING,
7     sk_tiempo_inicio     STRING,
8     sk_tiempo_fin        STRING,
9     sk_modalidad         STRING,
10    sk_ubicacion          STRING,
11    sk_estado             STRING,
12    sk_categoria          STRING,
13    valor_contrato        DOUBLE,
14    valor_facturado       DOUBLE,
15    valor_pagado          DOUBLE,
16    valor_pendiente_pago  DOUBLE,
17    dias_adicionados      INT,
18    cantidad_contratos    INT
19 )
20 PARTITIONED BY (anio_firma INT)
21 STORED AS PARQUET
22 """)
23
24 print("✅ Tabla de hechos 'hecho_contratos' creada (particionada por anio_firma).")
```

```
26/08/24 01:46:09 WARN HiveMetaStore: Location: file:/home/jovyan/work/spark-warehouse/secop_dw.db/hecho_contratos specified for non-external table:hecho_contratos
```

```
✅ Tabla de hechos 'hecho_contratos' creada (particionada por anio_firma).
```


5. Evidencia de la estructura del cubo

```
In [5]: 1 tablas_cubo = [
2       "hecho_contratos", "dim_entidad", "dim_proveedor", "dim_tiempo",
3       "dim_modalidad", "dim_ubicacion", "dim_estado_contrato", "dim_categoria",
4       ]
5
6 for tabla in tablas_cubo:
7     print(f"\n=== Estructura de {DB_NAME}.{tabla} ===")
8     spark.sql(f"DESCRIBE {DB_NAME}.{tabla}").show(truncate=False)
9
10 print(f"\nTablas creadas en '{DB_NAME}':")
11 spark.sql(f"SHOW TABLES IN {DB_NAME}").show(truncate=False)
```

```
=== Estructura de secop_dw.hecho_contratos ===
+-----+-----+-----+
|col_name|data_type|comment|
+-----+-----+-----+
|id_contrato|string|null|
|sk_entidad|string|null|
|sk_proveedor|string|null|
|sk_tiempo_firma|string|null|
|sk_tiempo_inicio|string|null|
|sk_tiempo_fin|string|null|
|sk_modalidad|string|null|
|sk_ubicacion|string|null|
|sk_estado|string|null|
|sk_categoria|string|null|
|valor_contrato|double|null|
|valor_facturado|double|null|
|valor_pagado|double|null|
|valor_pendiente_pago|double|null|
```

Conclusiones

- Se implementó fielmente el esquema estrellapropuesto en el taller: 7 dimensiones + 1 hecho.
- `dim_tiempo` se modeló como **dimensión de rol** (una sola tabla física, referenciada 3 veces desde el hecho), técnica estándar de modelado dimensional para evitar duplicar dimensiones idénticas.
- El cubo queda listo para ser poblado por `Transformacion.ipynb` y `Cargue.ipynb`.

Normalización al Modelo del Cubo (Plata)

Taller ETL – Cubo SECOP

Objetivo: Tomar los datos crudos del área **bronze** y normalizarlos exactamente en las 7 dimensiones + 1 hecho definidos en `CuboDatos.ipynb`, aplicando limpieza, tipado y generación de llaves. El resultado se persiste en el área **silver (plata)**.

Nota sobre `dim_tiempo` (dimensión de rol): el hecho referencia la misma dimensión de tiempo 3 veces (`sk_tiempo_firma`, `sk_tiempo_inicio`, `sk_tiempo_fin`). Para esto, `dim_tiempo` se construye a partir de la **unión de las 3 fechas** (firma, inicio, fin) — cada fecha distinta que aparezca en cualquiera de los 3 roles genera una sola fila en `dim_tiempo`, y las 3 llaves foráneas del hecho usan la misma función de hash sobre la fecha correspondiente, garantizando que apunten correctamente a esa fila compartida.

El notebook es robusto a columnas faltantes (se rellenan como nulas y se reportan).

El notebook es robusto a columnas faltantes (se rellenan como nulas y se reportan).

1. Sesión de Spark ¶

```
In [1]: 1 from pyspark.sql import SparkSession
2 from pyspark.sql import functions as F
3 from pyspark.sql.types import DoubleType, IntegerType
4
5 spark = (
6     SparkSession.builder
7     .appName("SECOP_Transformacion")
8     .master("local[*]")
9     .config("spark.driver.memory", "4g")
10    .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000")
11    .config("spark.sql.catalogImplementation", "hive")
12    .config("spark.sql.execution.arrow.pyspark.enabled", "true")
13    .enableHiveSupport()
14    .getOrCreate()
15 )
16
17 spark.sparkContext.setLogLevel("WARN")
18 print("✅ Sesión Spark inicializada:", spark.version)
```

✅ Sesión Spark inicializada: 3.1.2

2. Lectura del área Bronce

```
In [2]: 1 BRONZE_PATH = "hdfs://namenode:9000/datalake/bronze/secop/contratos"
2
3 df_bronze = spark.read.parquet(BRONZE_PATH)
4 print(f"Registros en bronze: {df_bronze.count()} | Columnas: {len(df_bronze.columns)}")
5 df_bronze.printSchema()
```

[Stage 1:> (0 + 8) / 8]

Registros en bronze: 20000 | Columnas: 88

```
root
|-- nombre_entidad: string (nullable = true)
|-- nit_entidad: string (nullable = true)
|-- departamento: string (nullable = true)
|-- ciudad: string (nullable = true)
|-- localizaci_n: string (nullable = true)
|-- orden: string (nullable = true)
|-- sector: string (nullable = true)
|-- rama: string (nullable = true)
|-- entidad_centralizada: string (nullable = true)
|-- proceso_de_compra: string (nullable = true)
|-- id_contrato: string (nullable = true)
|-- referencia_del_contrato: string (nullable = true)
|-- estado_contrato: string (nullable = true)
|-- codigo_de_categoria_principal: string (nullable = true)
|-- descripcion_del_proceso: string (nullable = true)
|-- tipo_de_contrato: string (nullable = true)
|-- modalidad_de_contratacion: string (nullable = true)
|-- justificacion_modalidad_de: string (nullable = true)
|-- fecha_de_firma: string (nullable = true)
|-- fecha_de_inicio_del_contrato: string (nullable = true)
|-- fecha_de_fin_del_contrato: string (nullable = true)
|-- condiciones_de_entrega: string (nullable = true)
|-- tipodocproveedor: string (nullable = true)
|-- documento_proveedor: string (nullable = true)
|-- proveedor_adjudicado: string (nullable = true)
|-- es_grupo: string (nullable = true)
|-- es_pyme: string (nullable = true)
|-- habilita_pago_adelantado: string (nullable = true)
|-- liquidaci_n: string (nullable = true)
|-- obligaci_n_ambiental: string (nullable = true)
|-- obligaciones_postconsumo: string (nullable = true)
|-- reversion: string (nullable = true)
|-- origen_de_los_recursos: string (nullable = true)
|-- destino_gasto: string (nullable = true)
|-- valor_del_contrato: string (nullable = true)
|-- valor_de_pago_adelantado: string (nullable = true)
|-- valor_facturado: string (nullable = true)
|-- valor_pendiente_de_pago: string (nullable = true)
|-- valor_pagado: string (nullable = true)
|-- valor_amortizado: string (nullable = true)
|-- valor_pendiente_de: string (nullable = true)
|-- valor_pendiente_de_ejecucion: string (nullable = true)
|-- saldo_cdp: string (nullable = true)
|-- saldo_vigencia: string (nullable = true)
|-- espostconflicto: string (nullable = true)
|-- dias_adicionados: string (nullable = true)
|-- puntos_del_acuerdo: string (nullable = true)
|-- pilares_del_acuerdo: string (nullable = true)
|-- urlproceso: string (nullable = true)
|-- nombre_representante_legal: string (nullable = true)
|-- nacionalidad_representante_legal: string (nullable = true)
|-- domicilio_representante_legal: string (nullable = true)
|-- tipo_de_identificaci_n_representante_legal: string (nullable = true)
|-- identificaci_n_representante_legal: string (nullable = true)
|-- g_nero_representante_legal: string (nullable = true)
|-- presupuesto_general_de_la_nacion_pgn: string (nullable = true)
|-- sistema_general_de_participaciones: string (nullable = true)
```

```
|-- sistema_general_de_participaciones: string (nullable = true)
|-- sistema_general_de_regal_as: string (nullable = true)
|-- recursos_propios_alcald_as_gobernaciones_y_resguardos_ind_genas_: string (nullable = true)
|-- recursos_de_credito: string (nullable = true)
|-- recursos_propios: string (nullable = true)
|-- ultima_actualizacion: string (nullable = true)
|-- codigo_entidad: string (nullable = true)
|-- codigo_proveedor: string (nullable = true)
|-- objeto_del_contrato: string (nullable = true)
|-- duraci_n_del_contrato: string (nullable = true)
|-- nombre_del_banco: string (nullable = true)
|-- tipo_de_cuenta: string (nullable = true)
|-- n_mero_de_cuenta: string (nullable = true)
|-- el_contrato_puede_ser_prorrogado: string (nullable = true)
|-- nombre_ordenador_del_gasto: string (nullable = true)
|-- tipo_de_documento_ordenador_del_gasto: string (nullable = true)
|-- n_mero_de_documento_ordenador_del_gasto: string (nullable = true)
|-- nombre_supervisor: string (nullable = true)
|-- tipo_de_documento_supervisor: string (nullable = true)
|-- n_mero_de_documento_supervisor: string (nullable = true)
|-- nombre_ordenador_de_pago: string (nullable = true)
|-- tipo_de_documento_ordenador_de_pago: string (nullable = true)
|-- n_mero_de_documento_ordenador_de_pago: string (nullable = true)
|-- documentos_tipo: string (nullable = true)
|-- descripcion_documentos_tipo: string (nullable = true)
|-- direcci_n_de_ejecuci_n_del_contrato: string (nullable = true)
|-- fecha_inicio_liquidacion: string (nullable = true)
|-- fecha_fin_liquidacion: string (nullable = true)
|-- fecha_de_notificaci_n_de_prorrogaci_n: string (nullable = true)
|-- _fecha_extraccion: string (nullable = true)
|-- _fuente: string (nullable = true)
|-- fecha_ingesta: date (nullable = true)
```

3. Mapeo fuente - modelo del cubo

```
In [3]: 1 MAPA_COLUMNNAS = {
2       "id_contrato": "id_contrato",
3       "codigo_entidad": "codigo_entidad",
4       "nit_entidad": "nit_entidad",
5       "nombre_entidad": "nombre_entidad",
6       "orden": "orden",
7       "sector": "sector",
8       "rama": "rama",
9       "entidad_centralizada": "centralizada",
10      "codigo_proveedor": "codigo_proveedor",
11      "tipodocproveedor": "tipo_documento",
12      "documento_proveedor": "documento_proveedor",
13      "proveedor_adjudicado": "nombre_proveedor",
14      "es_grupo": "es_grupo",
15      "es_pyme": "es_pyme",
16      "fecha_de_firma": "fecha_firma",
17      "fecha_de_inicio_del_contrato": "fecha_inicio",
18      "fecha_de_fin_del_contrato": "fecha_fin",
19      "modalidad_de_contratacion": "modalidad_contratacion",
20      "justificacion_modalidad_de_contratacion": "justificacion_modalidad",
21      "tipo_de_contrato": "tipo_contrato",
22      "condiciones_de_entrega": "condiciones_entrega",
23      "departamento": "departamento",
24      "ciudad": "ciudad",
25      "localizaci_n": "localizacion",
26      "estado_contrato": "estado_contrato",
27      "liquidaci_n": "liquidacion",
28      "reversion": "reversion",
29      "habilita_pago_adelantado": "habilita_pago_adelantado",
30      "codigo_de_categoria_principal": "codigo_categoria_principal",
31      "descripcion_del_proceso": "descripcion_proceso",
32      "objeto_del_contrato": "objeto_contrato",
33      "valor_del_contrato": "valor_contrato",
34      "valor_facturado": "valor_facturado",
35      "valor_pagado": "valor_pagado",
36      "valor_pendiente_de_pago": "valor_pendiente_pago",
37      "dias_adicionados": "dias_adicionados",
38  }
39
40  def safe_col(df, nombre_origen, alias):
41      if nombre_origen in df.columns:
42          return F.col(nombre_origen).alias(alias)
43      else:
44          return F.lit(None).cast("string").alias(alias)
45
46  faltantes = [c for c in MAPA_COLUMNNAS if c not in df_bronce.columns]
47  print(f"Columnas del mapa no encontradas en bronce (se rellenan como NULL): {len(faltantes)}")
48  if faltantes:
49      print(faltantes)
50
51  df_normalizado = df_bronce.select([
52      safe_col(df_bronce, origen, destino) for origen, destino in MAPA_COLUMNNAS.items()
53  ])
54
55  df_normalizado.printSchema()

Columnas del mapa no encontradas en bronce (se rellenan como NULL): 1
['justificacion_modalidad_de_contratacion']
root
|-- id_contrato: string (nullable = true)
|-- codigo_entidad: string (nullable = true)
|-- nit_entidad: string (nullable = true)
|-- nombre_entidad: string (nullable = true)
|-- orden: string (nullable = true)
|-- sector: string (nullable = true)
|-- rama: string (nullable = true)
|-- centralizada: string (nullable = true)
-----
```

```
-- orden: string (nullable = true)
-- sector: string (nullable = true)
-- rama: string (nullable = true)
-- centralizada: string (nullable = true)
-- codigo_proveedor: string (nullable = true)
-- tipo_documento: string (nullable = true)
-- documento_proveedor: string (nullable = true)
-- nombre_proveedor: string (nullable = true)
-- es_grupo: string (nullable = true)
-- es_pyme: string (nullable = true)
-- fecha_firma: string (nullable = true)
-- fecha_inicio: string (nullable = true)
-- fecha_fin: string (nullable = true)
-- modalidad_contratacion: string (nullable = true)
-- justificacion_modalidad: string (nullable = true)
-- tipo_contrato: string (nullable = true)
-- condiciones_entrega: string (nullable = true)
-- departamento: string (nullable = true)
-- ciudad: string (nullable = true)
-- localizacion: string (nullable = true)
-- estado_contrato: string (nullable = true)
-- liquidacion: string (nullable = true)
-- reversion: string (nullable = true)
-- habilita_pago_adelantado: string (nullable = true)
-- codigo_categoria_principal: string (nullable = true)
-- descripcion_proceso: string (nullable = true)
-- objeto_contrato: string (nullable = true)
-- valor_contrato: string (nullable = true)
-- valor_facturado: string (nullable = true)
-- valor_pagado: string (nullable = true)
-- valor_pendiente_pago: string (nullable = true)
-- dias_adicionados: string (nullable = true)
```

4. Limpieza y tipado

Se convierten variables a sus formatos más adecuados, como valores, fechas y días.

```
In [4]: 1 df_limpio = (
2         df_normalizado
3         .dropDuplicates(["id_contrato"])
4         .na.drop(subset=["id_contrato"])
5         .withColumn("fecha_firma", F.to_date("fecha_firma"))
6         .withColumn("fecha_inicio", F.to_date("fecha_inicio"))
7         .withColumn("fecha_fin", F.to_date("fecha_fin"))
8         .withColumn("dias_adicionados", F.col("dias_adicionados").cast(IntegerType()))
9         .withColumn("valor_contrato", F.col("valor_contrato").cast(DoubleType()))
10        .withColumn("valor_facturado", F.col("valor_facturado").cast(DoubleType()))
11        .withColumn("valor_pagado", F.col("valor_pagado").cast(DoubleType()))
12        .withColumn("valor_pendiente_pago", F.col("valor_pendiente_pago").cast(DoubleType()))
13        .filter(F.col("fecha_firma").isNotNull() & F.col("valor_contrato").isNotNull())
14    )
15
16 print(f"Registros después de limpieza: {df_limpio.count()}")
17 df_limpio.select("id_contrato", "estado_contrato", "valor_contrato", "fecha_firma").show(5, truncate=False)
```

Registros después de limpieza: 18581

[Stage 6:> (0 + 8) / 8]

id_contrato	estado_contrato	valor_contrato	fecha_firma
C01.PCCNTR.1004753	Cerrado	3.6391009979E8	2019-06-21
C01.PCCNTR.1334215	Cerrado	4.1855E7	2020-02-03
C01.PCCNTR.1358274	Modificado	3.878586833E7	2020-02-08
C01.PCCNTR.1415507	Aprobado	5.836315E7	2020-03-01
C01.PCCNTR.1444423	Aprobado	1.08E7	2020-03-13

only showing top 5 rows

5. Construcción de las 7 dimensiones

Cada dimensión se construye por sus valores únicos, con llave surrogate (`sk_*`) generada por `sha2` sobre su llave natural. `dim_tiempo` es especial: se construye a partir de la unión de las 3 columnas de fecha (firma, inicio, fin), ya que es una dimensión de rol compartida.

```
In [5]: 1 def construir_dimension(df, columnas_llave_natural, columnas_atributo, nombre_sk):
2     columnas = list(dict.fromkeys(columnas_llave_natural + columnas_atributo))
3     return (
4         df.select(*columnas)
5         .dropDuplicates(columnas_llave_natural)
6         .withColumn(nombre_sk, F.sha2(F.concat_ws("|", *[F.coalesce(F.col(c), F.lit("NA")) for c in columnas_llave_natural]
7         .select(nombre_sk, *columnas)
8     )
9
10 df_dim_entidad = construir_dimension(
11     df_limpio, ["codigo_entidad"],
12     ["nit_entidad", "nombre_entidad", "orden", "sector", "rama", "centralizada"],
13     "sk_entidad",
14 )
15
16 df_dim_proveedor = construir_dimension(
17     df_limpio, ["documento_proveedor"],
18     ["codigo_proveedor", "tipo_documento", "nombre_proveedor", "es_grupo", "es_pyme"],
19     "sk_proveedor",
20 )
21
22 df_dim_modalidad = construir_dimension(
23     df_limpio, ["modalidad_contratacion"],
24     ["justificacion_modalidad", "tipo_contrato", "condiciones_entrega"],
25     "sk_modalidad",
26 )
27
28 df_dim_ubicacion = construir_dimension(
29     df_limpio, ["departamento", "ciudad", "localizacion"], [], "sk_ubicacion"
30 )
31
32 df_dim_estado = construir_dimension(
33     df_limpio, ["estado_contrato"],
34     ["liquidacion", "reversion", "habilita_pago_adelantado"],
35     "sk_estado",
36 )
37
38 df_dim_categoria = construir_dimension(
39     df_limpio, ["codigo_categoria_principal"],
40     ["descripcion_proceso", "objeto_contrato"],
41     "sk_categoria",
42 )
43
44 # --- dim_tiempo: dimensión de rol, construida como UNIÓN de las 3 fechas ---
45 fechas_union = (
46     df_limpio.select(F.col("fecha_firma").alias("fecha"))
47     .unionByName(df_limpio.select(F.col("fecha_inicio").alias("fecha")))
48     .unionByName(df_limpio.select(F.col("fecha_fin").alias("fecha")))
49     .filter(F.col("fecha").isNotNull())
50     .dropDuplicates(["fecha"])
51 )
52
53 df_dim_tiempo = (
54     fechas_union
55     .withColumn("sk_tiempo", F.sha2(F.col("fecha").cast("string"), 256))
56     .withColumn("anio", F.year("fecha"))
57     .withColumn("semestre", F.when(F.month("fecha") <= 6, 1).otherwise(2))
58     .withColumn("trimestre", F.quarter("fecha"))
59     .withColumn("mes", F.month("fecha"))
60     .withColumn("nombre_mes", F.date_format("fecha", "MMM"))
61     .withColumn("dia", F.dayofmonth("fecha"))
```

dim_entidad: 2805 registros

dim_proveedor: 18001 registros

dim_tiempo: 3255 registros

dim_modalidad: 14 registros

dim_ubicacion: 611 registros

dim_estado_contrato: 9 registros

[Stage 36:=====>(199 + 1) / 200]

dim_categoria: 1471 registros

6. Construcción de la tabla de hechos

Las 3 llaves de tiempo (`sk_tiempo_firma`, `sk_tiempo_inicio`, `sk_tiempo_fin`) se calculan con el mismo hash usado en `dim_tiempo`, garantizando que apunten a las filas correctas de esa dimensión compartida. `cantidad_contratos` se fija en 1 (grano del hecho = un contrato por fila; queda como métrica que suma para agregaciones).

```
In [6]: 1 df_hechos = (  
2     df_limpio  
3     .withColumn("sk_entidad", F.sha2(F.coalesce(F.col("codigo_entidad"), F.lit("NA")), 256))  
4     .withColumn("sk_proveedor", F.sha2(F.coalesce(F.col("documento_proveedor"), F.lit("NA")), 256))  
5     .withColumn("sk_tiempo_firma", F.sha2(F.col("fecha_firma").cast("string"), 256))  
6     .withColumn("sk_tiempo_inicio", F.when(F.col("fecha_inicio").isNotNull(), F.sha2(F.col("fecha_inicio").cast("string"), 256)))  
7     .withColumn("sk_tiempo_fin", F.when(F.col("fecha_fin").isNotNull(), F.sha2(F.col("fecha_fin").cast("string"), 256)))  
8     .withColumn("sk_modalidad", F.sha2(F.coalesce(F.col("modalidad_contratacion"), F.lit("NA")), 256))  
9     .withColumn("sk_ubicacion", F.sha2(F.concat_ws("|", F.coalesce(F.col("departamento"), F.lit("NA")), F.coalesce(F.col("ciudad"), F.lit("NA")), F.lit("NA")), 256))  
10    .withColumn("sk_estado", F.sha2(F.coalesce(F.col("estado_contrato"), F.lit("NA")), 256))  
11    .withColumn("sk_categoria", F.sha2(F.coalesce(F.col("codigo_categoria_principal"), F.lit("NA")), 256))  
12    .withColumn("cantidad_contratos", F.lit(1).cast(IntegerType()))  
13    .withColumn("anio_firma", F.year("fecha_firma"))  
14    .select(  
15        "id_contrato", "sk_entidad", "sk_proveedor", "sk_tiempo_firma", "sk_tiempo_inicio", "sk_tiempo_fin",  
16        "sk_modalidad", "sk_ubicacion", "sk_estado", "sk_categoria",  
17        "valor_contrato", "valor_facturado", "valor_pagado", "valor_pendiente_pago",  
18        "dias_adicionados", "cantidad_contratos", "anio_firma",  
19    )  
20 )  
21  
22 print(f"hecho_contratos: {df_hechos.count()} registros")  
23 df_hechos.select("id_contrato", "valor_contrato", "sk_entidad", "anio_firma").show(5, truncate=False)
```

hecho_contratos: 18581 registros

[Stage 41:→ (0 + 8) / 8]

id_contrato	valor_contrato	sk_entidad	anio_firma
[CO1.PCCNTR.1004753]	3.6391009979E8	d58d94c553375acc2c155d57f65a4107abdec5d50c64b6db1166a9383604eb9c	2019
[CO1.PCCNTR.1334215]	4.1855E7	1e881e1e48f20343c00d6b623900702921aa4ec9c604140832e2f40ecf6d4134	2020
[CO1.PCCNTR.1358274]	3.878586833E7	d01134138abbc5971d13fe8ccd5b8c6409653a4efe1668f9aa4c4d16d54d1df6	2020
[CO1.PCCNTR.1415507]	5.836315E7	b3cd809a27095a60e91786beb8a0c17a2ca701804d3504e7164f375bf88bcc51	2020
[CO1.PCCNTR.1444423]	1.08E7	db332d69146a50fa907a08da6586e14348f3873eacfb844f04a5cab4f5e06e058	2020

only showing top 5 rows

7. Persistencia en el área Plata

```
In [7]: 1 SILVER_BASE = "hdfs://namenode:9000/datalake/silver/secop"
2
3 destinos = {
4     "hecho_contratos": df_hechos,
5     "dim_entidad": df_dim_entidad,
6     "dim_proveedor": df_dim_proveedor,
7     "dim_tiempo": df_dim_tiempo,
8     "dim_modalidad": df_dim_modalidad,
9     "dim_ubicacion": df_dim_ubicacion,
10    "dim_estado_contrato": df_dim_estado,
11    "dim_categoria": df_dim_categoria,
12 }
13
14 for nombre, df in destinos.items():
15     ruta = f"{SILVER_BASE}/{nombre}"
16     df.coalesce(4).write.mode("overwrite").parquet(ruta)
17     print(f"✓ {nombre} -> {ruta}")
```

26/08/24 01:58:23 WARN package: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.

✓ hecho_contratos -> hdfs://namenode:9000/datalake/silver/secop/hecho_contratos

✓ dim_entidad -> hdfs://namenode:9000/datalake/silver/secop/dim_entidad

✓ dim_proveedor -> hdfs://namenode:9000/datalake/silver/secop/dim_proveedor

✓ dim_tiempo -> hdfs://namenode:9000/datalake/silver/secop/dim_tiempo

✓ dim_modalidad -> hdfs://namenode:9000/datalake/silver/secop/dim_modalidad

✓ dim_ubicacion -> hdfs://namenode:9000/datalake/silver/secop/dim_ubicacion

✓ dim_estado_contrato -> hdfs://namenode:9000/datalake/silver/secop/dim_estado_contrato

[Stage 67:=====> (1 + 3) / 4]

✓ dim_categoria -> hdfs://namenode:9000/datalake/silver/secop/dim_categoria

Conclusiones

- Se normalizaron los datos crudos del área bronce al modelo exacto del cubo propuesto para este taller: 7 dimensiones + 1 hecho.
- `dim_tiempo` se implementó correctamente como **dimensión de rol**, construida por unión de las 3 fechas del contrato, con las 3 llaves foráneas del hecho apuntando consistentemente a ella mediante el mismo hash.
- Los resultados quedaron persistidos en el área plata, listos para `Cargue.ipynb`.

2. Lectura del área Bronce

```
In [2]: 1 BRONZE_PATH = "hdfs://namenode:9000/datalake/bronze/secop/contratos"
2
3 df_bronce = spark.read.parquet(BRONZE_PATH)
4 print(f"Registros en bronce: {df_bronce.count()} | Columnas: {len(df_bronce.columns)}")
5 df_bronce.printSchema()
```

[Stage 1:> (0 + 8) / 8]

Registros en bronce: 20000 | Columnas: 88

```
root
|-- nombre_entidad: string (nullable = true)
|-- nit_entidad: string (nullable = true)
|-- departamento: string (nullable = true)
|-- ciudad: string (nullable = true)
|-- localizaci_n: string (nullable = true)
|-- orden: string (nullable = true)
|-- sector: string (nullable = true)
|-- rama: string (nullable = true)
|-- entidad_centralizada: string (nullable = true)
|-- proceso_de_compra: string (nullable = true)
|-- id_contrato: string (nullable = true)
|-- referencia_del_contrato: string (nullable = true)
|-- estado_contrato: string (nullable = true)
|-- codigo_de_categoria_principal: string (nullable = true)
|-- descripcion_del_proceso: string (nullable = true)
|-- tipo_de_contrato: string (nullable = true)
|-- modalidad_de_contratacion: string (nullable = true)
|-- justificacion_modalidad_de: string (nullable = true)
|-- fecha_de_firma: string (nullable = true)
|-- fecha_de_inicio_del_contrato: string (nullable = true)
|-- fecha_de_fin_del_contrato: string (nullable = true)
|-- condiciones_de_entrega: string (nullable = true)
|-- tipodocproveedor: string (nullable = true)
|-- documento_proveedor: string (nullable = true)
|-- proveedor_adjudicado: string (nullable = true)
|-- es_grupo: string (nullable = true)
|-- es_pyme: string (nullable = true)
|-- habilita_pago_adelantado: string (nullable = true)
|-- liquidaci_n: string (nullable = true)
|-- obligaci_n_ambiental: string (nullable = true)
```

```

|-- origen_de_los_recursos: string (nullable = true)
|-- destino_gasto: string (nullable = true)
|-- valor_del_contrato: string (nullable = true)
|-- valor_de_pago_adelantado: string (nullable = true)
|-- valor_facturado: string (nullable = true)
|-- valor_pendiente_de_pago: string (nullable = true)
|-- valor_pagado: string (nullable = true)
|-- valor_amortizado: string (nullable = true)
|-- valor_pendiente_de: string (nullable = true)
|-- valor_pendiente_de_ejecucion: string (nullable = true)
|-- saldo_cdp: string (nullable = true)
|-- saldo_vigencia: string (nullable = true)
|-- espostconflicto: string (nullable = true)
|-- dias_adicionados: string (nullable = true)
|-- puntos_del_acuerdo: string (nullable = true)
|-- pilares_del_acuerdo: string (nullable = true)
|-- urlproceso: string (nullable = true)
|-- nombre_representante_legal: string (nullable = true)
|-- nacionalidad_representante_legal: string (nullable = true)
|-- domicilio_representante_legal: string (nullable = true)
|-- tipo_de_identificaci_n_representante_legal: string (nullable = true)
|-- identificaci_n_representante_legal: string (nullable = true)
|-- g_nero_representante_legal: string (nullable = true)
|-- presupuesto_general_de_la_nacion_pgn: string (nullable = true)
|-- sistema_general_de_participaciones: string (nullable = true)
|-- sistema_general_de_regal_as: string (nullable = true)
|-- recursos_propios_alcald_as_gobernaciones_y_resguardos_ind_genas_: string (nullable = true)
|-- recursos_de_credito: string (nullable = true)
|-- recursos_propios: string (nullable = true)
|-- ultima_actualizacion: string (nullable = true)
|-- codigo_entidad: string (nullable = true)
|-- codigo_proveedor: string (nullable = true)
|-- objeto_del_contrato: string (nullable = true)
|-- duraci_n_del_contrato: string (nullable = true)
|-- nombre_del_banco: string (nullable = true)
|-- tipo_de_cuenta: string (nullable = true)
|-- n_mero_de_cuenta: string (nullable = true)
|-- el_contrato_puede_ser_prorrogado: string (nullable = true)
|-- nombre_ordenador_del_gasto: string (nullable = true)
|-- tipo_de_documento_ordenador_del_gasto: string (nullable = true)
|-- n_mero_de_documento_ordenador_del_gasto: string (nullable = true)
|-- nombre_supervisor: string (nullable = true)
|-- tipo_de_documento_supervisor: string (nullable = true)
|-- n_mero_de_documento_supervisor: string (nullable = true)
|-- nombre_ordenador_de_pago: string (nullable = true)
|-- tipo_de_documento_ordenador_de_pago: string (nullable = true)
|-- n_mero_de_documento_ordenador_de_pago: string (nullable = true)
|-- documentos_tipo: string (nullable = true)
|-- descripcion_documentos_tipo: string (nullable = true)
|-- direcci_n_de_ejecuci_n_del_contrato: string (nullable = true)
|-- fecha_inicio_liquidacion: string (nullable = true)
|-- fecha_fin_liquidacion: string (nullable = true)
|-- fecha_de_notificaci_n_de_prorroga: string (nullable = true)
|-- fecha_de_notificaci_n_de_prorroga: string (nullable = true)
|-- _fecha_extraccion: string (nullable = true)
|-- _fuente: string (nullable = true)
|-- fecha_ingesta: date (nullable = true)

```

3. Mapeo fuente - modelo del cubo ¶

```
In [3]: 1 MAPA_COLUMNAS = {
2       "id_contrato": "id_contrato",
3       "codigo_entidad": "codigo_entidad",
4       "nit_entidad": "nit_entidad",
5       "nombre_entidad": "nombre_entidad",
6       "orden": "orden",
7       "sector": "sector",
8       "rama": "rama",
9       "entidad_centralizada": "centralizada",
10      "codigo_proveedor": "codigo_proveedor",
11      "tipodocproveedor": "tipo_documento",
12      "documento_proveedor": "documento_proveedor",
13      "proveedor_adjudicado": "nombre_proveedor",
14      "es_grupo": "es_grupo",
15      "es_pyme": "es_pyme",
16      "fecha_de_firma": "fecha_firma",
17      "fecha_de_inicio_del_contrato": "fecha_inicio",
18      "fecha_de_fin_del_contrato": "fecha_fin",
19      "modalidad_de_contratacion": "modalidad_contratacion",
20      "justificacion_modalidad_de_contratacion": "justificacion_modalidad",
21      "tipo_de_contrato": "tipo_contrato",
22      "condiciones_de_entrega": "condiciones_entrega",
23      "departamento": "departamento",
24      "ciudad": "ciudad",
25      "localizaci_n": "localizacion",
26      "estado_contrato": "estado_contrato",
27      "liquidaci_n": "liquidacion",
28      "reversion": "reversion",
29      "habilita_pago_adelantado": "habilita_pago_adelantado",
30      "codigo_de_categoria_principal": "codigo_categoria_principal",
31      "descripcion_del_proceso": "descripcion_proceso",
32      "objeto_del_contrato": "objeto_contrato",
33      "valor_del_contrato": "valor_contrato",
34      "valor_facturado": "valor_facturado",
35      "valor_pagado": "valor_pagado",
36      "valor_pendiente_de_pago": "valor_pendiente_pago",
37      "dias_adicionados": "dias_adicionados",
38  }
39
40  def safe_col(df, nombre_origen, alias):
41      if nombre_origen in df.columns:
42          return F.col(nombre_origen).alias(alias)
43      else:
44          return F.lit(None).cast("string").alias(alias)
45
46  faltantes = [c for c in MAPA_COLUMNAS if c not in df_bronce.columns]
47  print(f" Columnas del mapa no encontradas en bronce (se rellenan como NULL): {len(faltantes)}")
48  if faltantes:
49      print(faltantes)
50
51  df_normalizado = df_bronce.select([
52      safe_col(df_bronce, origen, destino) for origen, destino in MAPA_COLUMNAS.items()
53  ])
54
55  df_normalizado.printSchema()
```

Columnas del mapa no encontradas en bronce (se rellenan como NULL): 1
 ['justificacion_modalidad_de_contratacion']
 root
 |-- id_contrato: string (nullable = true)
 |-- codigo_entidad: string (nullable = true)
 |-- nit_entidad: string (nullable = true)
 |-- nombre_entidad: string (nullable = true)
 |-- orden: string (nullable = true)
 |-- sector: string (nullable = true)
 |-- rama: string (nullable = true)
 |-- centralizada: string (nullable = true)
 |-- codigo_proveedor: string (nullable = true)
 |-- tipo_documento: string (nullable = true)
 |-- documento_proveedor: string (nullable = true)
 |-- nombre_proveedor: string (nullable = true)
 |-- es_grupo: string (nullable = true)
 |-- es_pyme: string (nullable = true)
 |-- fecha_firma: string (nullable = true)
 |-- fecha_inicio: string (nullable = true)
 |-- fecha_fin: string (nullable = true)
 |-- modalidad_contratacion: string (nullable = true)
 |-- justificacion_modalidad: string (nullable = true)
 |-- tipo_contrato: string (nullable = true)
 |-- condiciones_entrega: string (nullable = true)
 |-- departamento: string (nullable = true)
 |-- ciudad: string (nullable = true)
 |-- localizacion: string (nullable = true)
 |-- estado_contrato: string (nullable = true)
 |-- liquidacion: string (nullable = true)
 |-- reversion: string (nullable = true)
 |-- habilita_pago_adelantado: string (nullable = true)
 |-- codigo_categoria_principal: string (nullable = true)
 |-- descripcion_proceso: string (nullable = true)
 |-- objeto_contrato: string (nullable = true)
 |-- valor_contrato: string (nullable = true)
 |-- valor_facturado: string (nullable = true)
 |-- valor_pagado: string (nullable = true)
 |-- valor_pendiente_pago: string (nullable = true)
 |-- dias_adicionados: string (nullable = true)

4. Limpieza y tipado

```
n [4]: 1 df_limpio = (  
2     df_normalizado  
3     .dropDuplicates(["id_contrato"])  
4     .na.drop(subset=["id_contrato"])  
5     .withColumn("fecha_firma", F.to_date("fecha_firma"))  
6     .withColumn("fecha_inicio", F.to_date("fecha_inicio"))  
7     .withColumn("fecha_fin", F.to_date("fecha_fin"))  
8     .withColumn("dias_adicionados", F.col("dias_adicionados").cast(IntegerType()))  
9     .withColumn("valor_contrato", F.col("valor_contrato").cast(DoubleType()))  
10    .withColumn("valor_facturado", F.col("valor_facturado").cast(DoubleType()))  
11    .withColumn("valor_pagado", F.col("valor_pagado").cast(DoubleType()))  
12    .withColumn("valor_pendiente_pago", F.col("valor_pendiente_pago").cast(DoubleType()))  
13    .filter(F.col("fecha_firma").isNotNull() & F.col("valor_contrato").isNotNull())  
14 )  
15  
16 print(f"Registros después de limpieza: {df_limpio.count()}")  
17 df_limpio.select("id_contrato", "estado_contrato", "valor_contrato", "fecha_firma").show(5, truncate=False)
```

Registros después de limpieza: 18581

[Stage 6:=====>

(3 + 5) / 8]

id_contrato	estado_contrato	valor_contrato	fecha_firma
C01.PCCNTR.1004753	Cerrado	3.6391009979E8	2019-06-21
C01.PCCNTR.1334215	Cerrado	4.1855E7	2020-02-03
C01.PCCNTR.1358274	Modificado	3.878586833E7	2020-02-08
C01.PCCNTR.1415507	Aprobado	5.836315E7	2020-03-01
C01.PCCNTR.1444423	Aprobado	1.08E7	2020-03-13

only showing top 5 rows

5. Construcción de las 7 dimensiones

Cada dimensión se construye a partir de una tabla (o tablas) con "raw" (datos crudos) y se transforma en una tabla con "fact" (datos hechos) y se almacena en una base de datos.

dim_entidad: 2005 registros

dim_proveedor: 18001 registros

dim_tiempo: 3255 registros

dim_modalidad: 14 registros

dim_ubicacion: 611 registros

dim_estado_contrato: 9 registros

[Stage 35:=====> (177 + 8) / 200]

dim_categoria: 1471 registros

6. Construcción de la tabla de hechos ¶

Las 3 llaves de tiempo (sk_tiempo_firma, sk_tiempo_inicio, sk_tiempo_fin) se calculan con el mismo hash usado en dim_tiempo, garantizando que apunten a las filas correctas de esa dimensión compartida. cantidad_contratos se fija en 1 (grano del hecho = un contrato por fila; queda como métrica que suma para agregaciones).

```
In [7]: 1 df_hechos = (  
2     df_limpio  
3     .withColumn("sk_entidad", F.sha2(F.coalesce(F.col("codigo_entidad"), F.lit("NA")), 256))  
4     .withColumn("sk_proveedor", F.sha2(F.coalesce(F.col("documento_proveedor"), F.lit("NA")), 256))  
5     .withColumn("sk_tiempo_firma", F.sha2(F.col("fecha_firma").cast("string"), 256))  
6     .withColumn("sk_tiempo_inicio", F.when(F.col("fecha_inicio").isNotNull(), F.sha2(F.col("fecha_inicio").cast("string"),  
7     .withColumn("sk_tiempo_fin", F.when(F.col("fecha_fin").isNotNull(), F.sha2(F.col("fecha_fin").cast("string"), 256)))  
8     .withColumn("sk_modalidad", F.sha2(F.coalesce(F.col("modalidad_contratacion"), F.lit("NA")), 256))  
9     .withColumn("sk_ubicacion", F.sha2(F.concat_ws("|", F.coalesce(F.col("departamento"), F.lit("NA")), F.coalesce(F.col("ciudad",  
10    .withColumn("sk_estado", F.sha2(F.coalesce(F.col("estado_contrato"), F.lit("NA")), 256))  
11    .withColumn("sk_categoria", F.sha2(F.coalesce(F.col("codigo_categoria_principal"), F.lit("NA")), 256))  
12    .withColumn("cantidad_contratos", F.lit(1).cast(IntegerType()))  
13    .withColumn("anio_firma", F.year("fecha_firma"))  
14    .select(  
15        "id_contrato", "sk_entidad", "sk_proveedor", "sk_tiempo_firma", "sk_tiempo_inicio", "sk_tiempo_fin",  
16        "sk_modalidad", "sk_ubicacion", "sk_estado", "sk_categoria",  
17        "valor_contrato", "valor_facturado", "valor_pagado", "valor_pendiente_pago",  
18        "dias_adicionados", "cantidad_contratos", "anio_firma",  
19    )  
20 )  
21  
22 print(f"hecho_contratos: {df_hechos.count()} registros")  
23 df_hechos.select("id_contrato", "valor_contrato", "sk_entidad", "anio_firma").show(5, truncate=False)
```

hecho_contratos: 18581 registros

id_contrato	valor_contrato	sk_entidad	anio_firma
C01.PCCNTR.1004753	3.63910099797E8	d58d94c553375acc2c155d57f65a4107abdec5d50c64b6db1166a9383604eb9c	2019
C01.PCCNTR.1334215	4.1855E7	1e881e1e48f20343c00d6b623900702921aa4ec9c604140832e2f40ecf6d4134	2020
C01.PCCNTR.1358274	3.878586833E7	d01134138abbc5971d13fe8ccd5b8c6409653a4efe1668f9aa4c4d16d54d1df6	2020
C01.PCCNTR.1415507	5.836315E7	b3cd809a27095a60e91786beb8a0c17a2ca701804d3504e7164f375bf88bcc51	2020

```
In [8]: 1 SILVER_BASE = "hdfs://namenode:9000/datalake/silver/secop"  
2  
3 destinos = {  
4     "hecho_contratos": df_hechos,  
5     "dim_entidad": df_dim_entidad,  
6     "dim_proveedor": df_dim_proveedor,  
7     "dim_tiempo": df_dim_tiempo,  
8     "dim_modalidad": df_dim_modalidad,  
9     "dim_ubicacion": df_dim_ubicacion,  
10    "dim_estado_contrato": df_dim_estado,  
11    "dim_categoria": df_dim_categoria,  
12 }  
13  
14 for nombre, df in destinos.items():  
15     ruta = f"{SILVER_BASE}/{nombre}"  
16     df.coalesce(4).write.mode("overwrite").parquet(ruta)  
17     print(f"✅ {nombre} -> {ruta}")
```

```

✓ hecho_contratos -> hdfs://namenode:9000/datalake/silver/secop/hecho_contratos

✓ dim_entidad -> hdfs://namenode:9000/datalake/silver/secop/dim_entidad

✓ dim_proveedor -> hdfs://namenode:9000/datalake/silver/secop/dim_proveedor

✓ dim_tiempo -> hdfs://namenode:9000/datalake/silver/secop/dim_tiempo

✓ dim_modalidad -> hdfs://namenode:9000/datalake/silver/secop/dim_modalidad

✓ dim_ubicacion -> hdfs://namenode:9000/datalake/silver/secop/dim_ubicacion

✓ dim_estado_contrato -> hdfs://namenode:9000/datalake/silver/secop/dim_estado_contrato
[Stage 67:=====> (3 + 1) / 4]
✓ dim_categoria -> hdfs://namenode:9000/datalake/silver/secop/dim_categoria

```

Conclusiones

- 1 - Se normalizaron los datos crudos del área bronce al modelo exacto del cubo propuesto para este taller: 7 dimensiones + 1 hecho.
- 2 - `dim\_tiempo` se implementó correctamente como ****dimensión de rol****, construida por unión de las 3 fechas del contrato, con las 3 llaves foráneas del hecho apuntando consistentemente a ella mediante el mismo hash.
- 3 - Los resultados quedaron persistidos en el área ****plata****, listos para `Cargue.ipynb`.

Cargue del Cubo de Datos (Oro)

Taller ETL – Cubo SECOP Autor: Jurani Zabala Hernandez

Objetivo: Cargar en las tablas Hive del cubo (secop_dw, creadas en CuboDatos.ipynb) los datos normalizados del área **SILVER (plata)** (generados en Transformacion.ipynb), y validar el resultado con consultas SQL sobre hecho_contratos y sus 7 dimensiones — incluyendo la dimensión de rol dim_tiempo, usada 3 veces..

1. Sesión de Spark con soporte Hive

```

In [2]: 1 from pyspark.sql import SparkSession
2
3 spark = (
4     SparkSession.builder
5     .appName("SECOP_Cargue")
6     .master("local[*]")
7     .config("spark.driver.memory", "4g")
8     .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000")
9     .config("spark.sql.catalogImplementation", "hive")
10    .enableHiveSupport()
11    .getOrCreate()
12 )
13
14 DB_NAME = "secop_dw"
15 spark.catalog.setCurrentDatabase(DB_NAME)
16 spark.sparkContext.setLogLevel("WARN")
17 print(f"✓ Sesión Spark inicializada. Base de datos activa: {DB_NAME}")
18
19 from pyspark.sql import SparkSession
20
21 print(f"✓ Sesión Spark inicializada. Base de datos activa: {DB_NAME}")
22
23 ✓ Sesión Spark inicializada. Base de datos activa: secop_dw

```


2. Lectura del área Plata

```
In [3]: 1 SILVER_BASE = "hdfs://namenode:9000/datalake/silver/secop"
2
3 tablas = [
4     "hecho_contratos", "dim_entidad", "dim_proveedor", "dim_tiempo",
5     "dim_modalidad", "dim_ubicacion", "dim_estado_contrato", "dim_categoria",
6 ]
7
8 dfs = {}
9 for tabla in tablas:
10     ruta = f"{SILVER_BASE}/{tabla}"
11     dfs[tabla] = spark.read.parquet(ruta)
12     print(f"{tabla}: {dfs[tabla].count()} registros leídos desde {ruta}")
```

hecho_contratos: 18581 registros leídos desde hdfs://namenode:9000/datalake/silver/secop/hecho_contratos
dim_entidad: 2005 registros leídos desde hdfs://namenode:9000/datalake/silver/secop/dim_entidad
dim_proveedor: 18001 registros leídos desde hdfs://namenode:9000/datalake/silver/secop/dim_proveedor
dim_tiempo: 3255 registros leídos desde hdfs://namenode:9000/datalake/silver/secop/dim_tiempo
dim_modalidad: 14 registros leídos desde hdfs://namenode:9000/datalake/silver/secop/dim_modalidad
dim_ubicacion: 611 registros leídos desde hdfs://namenode:9000/datalake/silver/secop/dim_ubicacion
dim_estado_contrato: 9 registros leídos desde hdfs://namenode:9000/datalake/silver/secop/dim_estado_contrato
dim_categoria: 1471 registros leídos desde hdfs://namenode:9000/datalake/silver/secop/dim_categoria

3. Cargue en las tablas Hive del cubo

```
In [4]: 1 spark.conf.set("hive.exec.dynamic.partition", "true")
2 spark.conf.set("hive.exec.dynamic.partition.mode", "nonstrict")
3
4 dimensiones = [
5     "dim_entidad", "dim_proveedor", "dim_tiempo",
6     "dim_modalidad", "dim_ubicacion", "dim_estado_contrato", "dim_categoria",
7 ]
8
9 for tabla in dimensiones:
10     dfs[tabla].createOrReplaceTempView(f"tmp_{tabla}")
11     spark.sql(f"INSERT OVERWRITE TABLE {DB_NAME}.{tabla} SELECT * FROM tmp_{tabla}")
12     print(f"✅ {tabla} cargada en Hive.")
```

✅ dim_entidad cargada en Hive.

✅ dim_proveedor cargada en Hive.

✅ dim_tiempo cargada en Hive.

✅ dim_modalidad cargada en Hive.
✅ dim_ubicacion cargada en Hive.
✅ dim_estado_contrato cargada en Hive.

✅ dim_categoria cargada en Hive.

```
In [5]: 1 dfs["hecho_contratos"].createOrReplaceTempView("tmp_hecho_contratos")
2
3 columnas_hecho_sin_partition = [c for c in dfs["hecho_contratos"].columns if c != "anio_firma"]
4 select_cols = ", ".join(columnas_hecho_sin_partition)
5
6 spark.sql(f"""
7 INSERT OVERWRITE TABLE {DB_NAME}.hecho_contratos
8     PARTITION (anio_firma)
9 SELECT {select_cols}, anio_firma
10 FROM tmp_hecho_contratos
11 """)
12
13 print("✅ hecho_contratos cargada en Hive (particionada por anio_firma).")
```

26/08/24 02:15:54 WARN SessionState: METASTORE_FILTER_HOOK will be ignored, since hive.security.authorization.manager is set to instance of HiveAuthorizerFactory.

26/08/24 02:15:54 WARN HiveConf: HiveConf of name hive.internal.ss.authz.settings.applied.marker does not exist

26/08/24 02:15:54 WARN HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist

26/08/24 02:15:54 WARN HiveConf: HiveConf of name hive.stats.retries.wait does not exist

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updating partition stats fast for: hecho_contratos

26/08/24 02:16:14 WARN log: Updated size to 60943

26/08/24 02:16:14 WARN log: Updated size to 518072

26/08/24 02:16:14 WARN log: Updated size to 361004

26/08/24 02:16:14 WARN log: Updated size to 592859

26/08/24 02:16:14 WARN log: Updated size to 182197

26/08/24 02:16:14 WARN log: Updated size to 785780

26/08/24 02:16:14 WARN log: Updated size to 862852

26/08/24 02:16:14 WARN log: Updated size to 698459

26/08/24 02:16:14 WARN log: Updated size to 608377

26/08/24 02:16:14 WARN log: Updated size to 35666

26/08/24 02:16:14 WARN log: Updated size to 184963

✅ hecho_contratos cargada en Hive (particionada por anio_firma).

4. Validación del cargue: conteo de registros

```
In [6]: 1 todas_las_tablas = ["hecho_contratos"] + dimensiones
2 for tabla in todas_las_tablas:
3     total = spark.sql(f"SELECT COUNT(*) AS total FROM {DB_NAME}.{tabla}").collect()[0]["total"]
4     print(f"{tabla}: {total} registros en Hive")
```

hecho_contratos: 18581 registros en Hive
dim_entidad: 2005 registros en Hive
dim_proveedor: 18001 registros en Hive
dim_tiempo: 3255 registros en Hive
dim_modalidad: 14 registros en Hive
dim_ubicacion: 611 registros en Hive
dim_estado_contrato: 9 registros en Hive
dim_categoria: 1471 registros en Hive

5. Consultas de validación al cubo

Incluyen específicamente el uso de `dim_tiempo` en sus 3 roles distintos (firma, inicio, fin) mediante *joins* separados a la misma tabla física.

In [16]:

```
1 # Valor total contratado y cantidad de contratos por año de firma
2 spark.sql(f"""
3 SELECT anio_firma, COUNT(*) AS cantidad_contratos, format_number(SUM(valor_contrato), 2) AS valor_total
4 FROM {DB_NAME}.hecho_contratos
5 GROUP BY anio_firma
6 ORDER BY anio_firma
7 """).show()
```

[Stage 68:=====> (5 + 3) / 8]

anio_firma	cantidad_contratos	valor_total
2016	6	318,780,696.00
2017	79	7,089,044,600.34
2018	472	132,852,538,760.01
2019	463	116,012,621,772.74
2020	1167	131,793,601,887.58
2021	1880	324,659,589,495.97
2022	2365	225,604,928,239.34
2023	2816	305,039,409,443.37
2024	3220	244,028,535,929.90
2025	3544	532,179,550,632.98
2026	2569	317,040,479,793.21

In [17]:

```
1 # Top 10 entidades por valor contratado (hecho + entidad)
2 spark.sql(f"""
3 SELECT e.nombre_entidad, e.sector, COUNT(*) AS cantidad_contratos, format_number(SUM(h.valor_contrato), 2) AS valor_total
4 FROM {DB_NAME}.hecho_contratos h
5 JOIN {DB_NAME}.dim_entidad e ON h.sk_entidad = e.sk_entidad
6 GROUP BY e.nombre_entidad, e.sector
7 ORDER BY valor_total DESC
8 LIMIT 10
9 """).show(truncate=False)
```

[Stage 71:=====> (1 + 7) / 8]

nombre_entidad	sector	cantidad_contratos	valor_total
PARQUES NACIONALES NATURALES DE COLOMBIA - DIRECCION TERRITORIAL CARIBE			
Ambiente y Desarrollo Sostenible	8		99,831,459.00
TRANSITO DE LOS PATIOS			
Transporte	7		99,144,000.00
MUNICIPIO DE PUERTO ASIS PUTUMAYO			
Servicio Público	11		982,087,494.00
EMPRESA DE DESARROLLO URBANO DE MEDELLIN			
Vivienda, Ciudad y Territorio	5		981,625,408.00
ALCALDIA DE SAN PABLO - BOLIVAR			
Servicio Público	2		98,766,214.00
INSTITUTO PARA EL FOMENTO DEL DEPORTE, LA RECREACION, EL APROVECHAMIENTO DEL TIEMPO LIBRE Y LA EDUCACION FISICA DE BARRANCABER MEJA			
Servicio Público	9		98,666,667.00
CORPORACION PARA EL DESARROLLO SOSTENIBLE DEL ARCHIPIELAGO DE SAN ANDRES PROVIDENCIA Y SANTA CATALINA - CORALINA			
Ambiente y Desarrollo Sostenible	7		98,469,370.00
IUOC			
Educación Nacional	5		97,873,888.00
PERSONERIA MUNICIPAL GUARNE ANTIOQUIA			
Servicio Público	2		97,850,000.00
ARC-BACAIM6			
defensa	2		97,240,600.00

```
In [18]: 1 # dim_tiempo en sus 3 ROLES: duración real (fin - inicio) por modalidad, usando 2 joins distintos a dim_tiempo
2 spark.sql(f"""
3 SELECT
4     m.modalidad_contratacion,
5     COUNT(*) AS cantidad_contratos,
6     ROUND(AVG(DATEDIFF(t_fin.fecha, t_inicio.fecha)), 1) AS duracion_promedio_dias
7 FROM {DB_NAME}.hecho_contratos h
8 JOIN {DB_NAME}.dim_modalidad m      ON h.sk_modalidad = m.sk_modalidad
9 JOIN {DB_NAME}.dim_tiempo t_inicio ON h.sk_tiempo_inicio = t_inicio.sk_tiempo
10 JOIN {DB_NAME}.dim_tiempo t_fin   ON h.sk_tiempo_fin   = t_fin.sk_tiempo
11 GROUP BY m.modalidad_contratacion
12 ORDER BY duracion_promedio_dias DESC
13 """).show(truncate=False)
```

[Stage 76:=====> (2 + 6) / 8]

modalidad_contratacion	cantidad_contratos	duracion_promedio_dias
Licitación Pública Acuerdo Marco de Precios	1	2248.0
Concurso de méritos abierto	38	543.3
Licitación pública Obra Publica	31	391.8
Selección Abreviada Menor Cuantía Sin Manifestación Interés	6	291.2
Licitación pública	34	285.0
Contratación Directa (con ofertas)	241	280.1
Contratación régimen especial (con ofertas)	154	235.1
Contratación directa	13982	186.1
Selección Abreviada de Menor Cuantía	188	179.2
Contratación régimen especial	2434	171.6
Selección abreviada subasta inversa	167	150.4
Mínima cuantía	1048	117.5
Enajenación de bienes con subasta	1	34.0
Enajenación de bienes con sobre cerrado	1	30.0

```
In [11]: 1 # Distribución por ubicación y estado del contrato (hecho + ubicacion + estado)
2 spark.sql(f"""
3 SELECT u.departamento, es.estado_contrato, COUNT(*) AS cantidad_contratos, ROUND(SUM(h.valor_contrato), 2) AS valor_total
4 FROM {DB_NAME}.hecho_contratos h
5 JOIN {DB_NAME}.dim_ubicacion u      ON h.sk_ubicacion = u.sk_ubicacion
6 JOIN {DB_NAME}.dim_estado_contrato es ON h.sk_estado = es.sk_estado
7 GROUP BY u.departamento, es.estado_contrato
8 ORDER BY valor_total DESC
9 LIMIT 15
10 """).show(truncate=False)
```

[Stage 60:=====> (1 + 7) / 8]

departamento	estado_contrato	cantidad_contratos	valor_total
Distrito Capital de Bogotá	Modificado	1579	5.5722178595106E11
Distrito Capital de Bogotá	En ejecución	1590	2.2621986298531E11
Distrito Capital de Bogotá	terminado	653	1.8876253369355E11
Antioquia	Modificado	247	1.3585824997273E11
Antioquia	En ejecución	593	1.233639269952E11
Distrito Capital de Bogotá	Cerrado	1960	1.0326108707407E11
Antioquia	terminado	278	5.968389820572E10
Atlántico	Modificado	128	5.899031065363E10
Valle del Cauca	terminado	177	4.174343972351E10
Valle del Cauca	Modificado	272	3.906144546065E10
Cundinamarca	Aprobado	21	3.836020415847E10
Valle del Cauca	En ejecución	595	3.754363605071E10
Antioquia	Cerrado	627	3.151754295266E10
Distrito Capital de Bogotá	Aprobado	288	2.324736419935E10
Córdoba	Modificado	29	2.2450837791E10

```
In [19]: 1 # Categoría del proceso + proveedor (hecho + categoría + proveedor)
2 spark.sql(f"""
3 SELECT c.codigo_categoria_principal, p.es_pyme, COUNT(*) AS cantidad_contratos, format_number(SUM(h.valor_contrato), 2) AS v
4 FROM {DB_NAME}.hecho_contratos h
5 JOIN {DB_NAME}.dim_categoria c ON h.sk_categoria = c.sk_categoria
6 JOIN {DB_NAME}.dim_proveedor p ON h.sk_proveedor = p.sk_proveedor
7 GROUP BY c.codigo_categoria_principal, p.es_pyme
8 ORDER BY valor_total DESC
9 LIMIT 15
10 """).show(truncate=False)
```

[Stage 80:=====] (1 + 7) / 8]

codigo_categoria_principal	es_pyme	cantidad_contratos	valor_total
V1.92101602	No	4	994,164,331.00
V1.70131700	No	12	992,966,390.00
V1.93131610	No	1	991,565,351.00
V1.39131700	Si	1	99,900,000.00
V1.52152000	Si	1	99,841,642.00
V1.44103103	Si	3	99,784,387.00
V1.72102902	No	1	99,470,000.00
V1.81141902	No	2	99,460,000.00
V1.85121613	No	3	99,441,000.00
V1.81101600	No	2	99,267,000.00
V1.80141607	Si	9	985,769,700.00
V1.42142600	Si	2	98,768,032.00
V1.85101502	No	3	98,132,000.00
V1.51142400	No	1	98,000,000.00
V1.78131603	Si	1	976,941,700.00

Conclusiones

- Las 7 dimensiones y hecho_contratos fueron cargadas exitosamente desde plata, con el hecho particionado por anio_firma.
- Las consultas de validación confirman la integridad referencial del modelo, incluyendo el uso correcto de dim_tiempo como dimensión de rol (2 joins distintos a la misma tabla para calcular duración real de los contratos).
- Con esto se cierra el flujo ETL completo del taller.

Explotación Analítica del Cubo SECOP

Taller ETL – Cubo SECOP Autor: Jurani Zabala Hernandez

Objetivo: Explotar el cubo secop_dw (poblado por Cargue.ipynb) con consultas SQL de negocio, distintas a las de validación de Cargue.ipynb.

1. Sesión de Spark con soporte Hive

```
In [1]: 1 from pyspark.sql import SparkSession
2
3 spark = (
4     SparkSession.builder
5     .appName("SECOP_ConultasSQL")
6     .master("local[*]")
7     .config("spark.driver.memory", "4g")
8     .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000")
9     .config("spark.sql.catalogImplementation", "hive")
10    .enableHiveSupport()
11    .getOrCreate()
12 )
13
14 DB_NAME = "secop_dw"
15 spark.catalog.setCurrentDatabase(DB_NAME)
16 spark.sparkContext.setLogLevel("WARN")
17 print(f"✅ Sesión Spark inicializada. Base de datos activa: {DB_NAME}")
18 spark.sql(f"SHOW TABLES IN {DB_NAME}").show(truncate=False)
```

WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.apache.spark.unsafe.Platform (file:/usr/local/spark-3.1.2-bin-hadoop3.2/jars/spark-unsafe_2.12-3.1.2.jar) to constructor java.nio.DirectByteBuffer(long,int)
WARNING: Please consider reporting this to the maintainers of org.apache.spark.unsafe.Platform
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
26/08/24 02:34:47 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/08/24 02:34:54 WARN HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
26/08/24 02:34:54 WARN HiveConf: HiveConf of name hive.stats.retries.wait does not exist
26/08/24 02:35:09 WARN ObjectStore: Version information not found in metastore. hive.metastore.schema.validation is not enabled so recording the schema version 2.3.0
26/08/24 02:35:09 WARN ObjectStore: setMetaStoreSchemaVersion called but recording version is disabled: version = 2.3.0, comment = Set by MetaStore UNKNOWN@172.18.0.6
26/08/24 02:35:09 WARN ObjectStore: Failed to get database global_temp, returning NoSuchObjectException

✅ Sesión Spark inicializada. Base de datos activa: secop_dw

database	tableName	isTemporary
secop_dw	dim_categoria	false
secop_dw	dim_entidad	false
secop_dw	dim_estado_contrato	false
secop_dw	dim_modalidad	false
secop_dw	dim_proveedor	false
secop_dw	dim_tiempo	false
secop_dw	dim_ubicacion	false
secop_dw	hecho_contratos	false

2. Panorama general del cubo

```
In [3]: 1 spark.sql(f"""
2 SELECT
3     COUNT(*) AS total_contratos,
4     format_number(SUM(valor_contrato), 2) AS valor_total_contratado,
5     format_number(AVG(valor_contrato), 2) AS valor_promedio_contrato,
6     format_number(SUM(valor_pendiente_pago), 2) AS total_pendiente_pago
7 FROM {DB_NAME}.hecho_contratos
8 """).show(truncate=False)
```

[Stage 2:=====> (1 + 7) / 8]

total_contratos	valor_total_contratado	valor_promedio_contrato	total_pendiente_pago
18581	2,336,619,081,251.44	125,753,139.30	1,620,542,533,140.00

3. Evolución mensual del valor contratado (usando `dim_tiempo` en su rol de fecha de firma)

```
In [5]: 1 spark.sql(f"""
2 SELECT t.anio, t.mes, t.nombre_mes, COUNT(*) AS cantidad_contratos, format_number(SUM(h.valor_contrato), 2) AS valor_total
3 FROM {DB_NAME}.hecho_contratos h
4 JOIN {DB_NAME}.dim_tiempo t ON h.sk_tiempo_firma = t.sk_tiempo
5 GROUP BY t.anio, t.mes, t.nombre_mes
6 ORDER BY t.anio, t.mes
7 """).show(30, truncate=False)
```

[Stage 9:=====> (171 + 16) / 200]

anio	mes	nombre_mes	cantidad_contratos	valor_total
2016	1	January	1	38,640,000.00
2016	7	July	1	21,948,384.00
2016	8	August	2	51,180,152.00
2016	9	September	1	80,012,160.00
2016	12	December	1	127,000,000.00
2017	1	January	5	351,393,450.00
2017	2	February	4	194,135,363.00
2017	3	March	6	187,570,566.67
2017	4	April	2	49,200,000.00
2017	5	May	6	106,149,563.00
2017	6	June	1	6,780,000.00
2017	7	July	1	56,221,000.00
2017	8	August	5	288,625,100.00
2017	9	September	7	718,757,949.00
2017	10	October	6	369,947,321.00
2017	11	November	15	3,477,837,977.00
2017	12	December	21	1,282,426,310.67
2018	1	January	126	13,077,605,365.19
2018	2	February	31	6,717,147,012.70
2018	3	March	9	285,472,483.00
2018	4	April	14	734,975,396.58
2018	5	May	13	377,983,761.49
2018	6	June	25	1,748,078,002.40
2018	7	July	47	7,373,499,918.83
2018	8	August	44	3,020,407,820.52
2018	9	September	24	46,100,480,188.66
2018	10	October	48	14,285,927,463.50
2018	11	November	49	2,581,935,342.74
2018	12	December	42	36,549,026,004.40
2019	1	January	132	5,191,660,314.67

only showing top 30 rows

4. Concentración del gasto: top 10 entidades y su % del total

```
In [9]: 1 spark.sql(f"""
2 WITH por_entidad AS (
3     SELECT e.nombre_entidad, SUM(h.valor_contrato) AS valor_entidad
4     FROM {DB_NAME}.hecho_contratos h
5     JOIN {DB_NAME}.dim_entidad e ON h.sk_entidad = e.sk_entidad
6     GROUP BY e.nombre_entidad
7 ),
8 total AS (SELECT SUM(valor_entidad) AS valor_total FROM por_entidad)
9 SELECT
10     p.nombre_entidad,
11     format_number(p.valor_entidad, 2) AS valor_entidad,
12     format_number(100 * p.valor_entidad / t.valor_total, 2) AS porcentaje_del_total
13 FROM por_entidad p, total t
14 ORDER BY p.valor_entidad DESC
15 LIMIT 10
16 """).show(truncate=False)
```

26/08/24 02:42:23 WARN HiveConf: HiveConf of name hive.internal.ss.authz.settings.applied.marker does not exist
26/08/24 02:42:23 WARN HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
26/08/24 02:42:23 WARN HiveConf: HiveConf of name hive.stats.retries.wait does not exist

nombre_entidad	valor_entidad	porcentaje_del_total
DISTRITO ESPECIAL DE CIENCIA TECNOLOGIA E INNOVACION DE MEDELLIN	159,042,883,456.00	6.81
INVIAS	127,885,117,423.34	5.47
JEP	63,042,798,203.00	2.70
DISTRITO ESPECIAL INDUSTRIAL Y PORTUARIO DE BARRANQUILLA	59,508,583,079.00	2.55
IDRD - ENTIDAD OFICIAL.	52,473,508,828.00	2.25
UNP	49,502,462,037.00	2.12
UARIV-UNIDAD PARA LA ATENCION Y REPARACION INTEGRAL A LAS VICTIMAS	48,137,866,562.00	2.06
DILOF	45,183,171,995.15	1.93
MUNICIPIO DE TOCANCIPA	38,824,846,845.00	1.66
MUNICIPIO DE CAREPA	32,368,252,170.00	1.39

5. Modalidad de contratación por sector

```
In [11]: 1 spark.sql("""
2 SELECT e.sector, m.modalidad_contratacion, COUNT(*) AS cantidad_contratos, format_number(SUM(h.valor_contrato), 2) AS valor_
3 FROM {DB_NAME}.hecho_contratos h
4 JOIN {DB_NAME}.dim_entidad e ON h.sk_entidad = e.sk_entidad
5 JOIN {DB_NAME}.dim_modalidad m ON h.sk_modalidad = m.sk_modalidad
6 GROUP BY e.sector, m.modalidad_contratacion
7 ORDER BY e.sector, valor_total DESC
8 """).show(30, truncate=False)
```

[Stage 42:=====]

(1 + 7) / 8]

sector	modalidad_contratacion	cantidad_contratos	valor_total
Ambiente y Desarrollo Sostenible	Minima cuantía	41	883,389,076.13
Ambiente y Desarrollo Sostenible	Contratación Directa (con ofertas)	11	7,544,973,705.
Ambiente y Desarrollo Sostenible	Contratación directa	733	30,061,515,13
Ambiente y Desarrollo Sostenible	Licitación pública	3	3,218,489,460.
Ambiente y Desarrollo Sostenible	Contratación régimen especial (con ofertas)	3	2,965,475,714.
Ambiente y Desarrollo Sostenible	Contratación régimen especial	15	2,013,018,856.
Ambiente y Desarrollo Sostenible	Selección Abreviada Menor Cuantía Sin Manifestacion Interes	1	1,957,058,222.
Ambiente y Desarrollo Sostenible	Selección Abreviada de Menor Cuantía	12	1,682,715,910.
Ambiente y Desarrollo Sostenible	Concurso de méritos abierto	4	1,347,669,105.
Ambiente y Desarrollo Sostenible	Selección abreviada subasta inversa	6	1,015,381,089.
Ciencia Tecnología	Contratación Directa (con ofertas)	6	8,882,777,610.
Ciencia Tecnología	Contratación régimen especial	8	244,371,621.00
Ciencia Tecnología	Contratación directa	34	1,611,608,013.
Cultura	Contratación Directa (con ofertas)	8	716,710,817.02
Cultura	Licitación pública Obra Publica	3	47,100,155,99
Cultura	Minima cuantía	21	405,322,967.14
Cultura	Selección abreviada subasta inversa	2	3,545,638,159.
Cultura	Contratación directa	433	23,273,631,99
Cultura	Selección Abreviada de Menor Cuantía	5	2,307,616,183.
Cultura	Contratación régimen especial	27	1,362,023,292.
Cultura	Contratación régimen especial (con ofertas)	4	1,170,744,864.
Educación Nacional	Contratación régimen especial (con ofertas)	17	8,373,688,234.
Educación Nacional	Contratación régimen especial	70	7,622,262,359.
Educación Nacional	Contratación Directa (con ofertas)	6	4,620,276,394.
Educación Nacional	Licitación pública	2	36,276,455,41
Educación Nacional	Contratación directa	876	32,036,160,07

6. Participación de proveedores PYME en el valor contratado

```
In [13]: 1 spark.sql("""
2 SELECT
3     p.es_pyme,
4     COUNT(DISTINCT h.sk_proveedor) AS proveedores_distintos,
5     COUNT(*) AS cantidad_contratos,
6     format_number(SUM(h.valor_contrato), 2) AS valor_total
7 FROM {DB_NAME}.hecho_contratos h
8 JOIN {DB_NAME}.dim_proveedor p ON h.sk_proveedor = p.sk_proveedor
9 GROUP BY p.es_pyme
10 """).show(truncate=False)
```

	es_pyme	proveedores_distintos	cantidad_contratos	valor_total
No	15499	15838	1,630,416,479,870.37	
Si	2502	2743	706,202,601,381.07	

7. Cumplimiento de pagos: contratos con mayor % pendiente

```
In [17]: 1 spark.sql("""
2 SELECT
3     e.nombre_entidad, h.id_contrato, format_number(h.valor_contrato,2) as valor_contrato, format_number(h.valor_pagado,2) as
4     format_number(100 * h.valor_pendiente_pago / NULLIF(h.valor_contrato, 0), 2) AS porcentaje_pendiente
5 FROM {DB_NAME}.hecho_contratos h
6 JOIN {DB_NAME}.dim_entidad e ON h.sk_entidad = e.sk_entidad
7 WHERE h.valor_pendiente_pago IS NOT NULL AND h.valor_contrato > 0
8 ORDER BY porcentaje_pendiente DESC
9 LIMIT 15
10 """).show(truncate=False)
```

[Stage 86:=====]

(1 + 7) / 8]

nombre_entidad	id_contrato	valor_contrato	valor_pagado	valor_pendiente_pago
porcentaje_pendiente				
EMPRESA SOCIAL DEL ESTADO DEL MUNICIPIO DE VILLAVICENCIO	C01.PCCNTR.9382145	3,150,000,000.00	3,150,000.00	3,146,850,000.00
99.90				
SECRETARIA DISTRITAL DE SEGURIDAD, CONVIVENCIA Y JUSTICIA	C01.PCCNTR.5147253	4,204,125,000.00	16,837,913.00	4,187,287,087.00
99.60				
Unidad de Búsqueda de Personas Desaparecidas	C01.PCCNTR.9561451	2,000,000,000.00	17,919,985.00	1,982,080,014.00
99.10				
INSTITUCION UNIVERSITARIA DE BARRANQUILLA-IUB	C01.PCCNTR.3628187	10,240,000.00	156,100.00	10,083,900.00
98.48				
(Secretaría Distrital de Integración Social)	C01.PCCNTR.9618490	17,851,500.00	297,525.00	17,553,975.00
98.33				
AGENCIA NACIONAL DE HIDROCARBUROS**	C01.PCCNTR.5128186	20,819,759.00	452,603.00	20,367,156.00
97.83				
AGENCIA NACIONAL DE HIDROCARBUROS**	C01.PCCNTR.8975146	60,300,000.00	1,563,333.00	58,736,667.00
97.41				
INSTITUTO DE DEPORTES Y RECREACION DE MEDELLIN	C01.PCCNTR.5005821	25,503,573.00	732,160.00	24,771,413.00
97.13				
SENA REGIONAL CASANARE Grupo Administrativo Mixto	C01.PCCNTR.8962148	49,993,935.00	1,465,024.00	48,528,911.00
97.07				
IDARTES	C01.PCCNTR.7251801	60,996,000.00	1,794,000.00	59,202,000.00
97.06				
MINCIENCIAS	C01.PCCNTR.6566510	49,083,333.00	1,583,333.00	47,500,000.00
96.77				
ALCALDIA MUNICIPAL COTA	C01.PCCNTR.4784705	28,050,000.00	990,000.00	27,060,000.00
96.47				
DANE - DIRECCION TERRITORIAL CENTRO	C01.PCCNTR.9638399	8,223,500.00	293,400.00	7,930,100.00
96.43				
ALCALDIA DE MOCOA	C01.PCCNTR.9123749	21,340,000.00	776,000.00	20,564,000.00
96.36				
FISCALIA GENERAL DE LA NACION REGIONAL CENTROSUR	C01.PCCNTR.2039949	756,245,984.00	27,627,014.00	728,618,970.00
96.35				

8. Duración real del contrato (fin – inicio) por modalidad — usa `dim_tiempo` en 2 roles distintos ¶

In [18]:

```
1 spark.sql(f"""
2 SELECT
3     m.modalidad_contratacion,
4     COUNT(*) AS cantidad_contratos,
5     ROUND(AVG(DATEDIFF(t_fin.fecha, t_inicio.fecha)), 1) AS duracion_promedio_dias,
6     ROUND(AVG(h.dias_adicionados), 2) AS promedio_dias_adicionados
7 FROM {DB_NAME}.hecho_contratos h
8 JOIN {DB_NAME}.dim_modalidad m      ON h.sk_modalidad = m.sk_modalidad
9 JOIN {DB_NAME}.dim_tiempo t_inicio ON h.sk_tiempo_inicio = t_inicio.sk_tiempo
10 JOIN {DB_NAME}.dim_tiempo t_fin   ON h.sk_tiempo_fin   = t_fin.sk_tiempo
11 WHERE h.dias_adicionados IS NOT NULL
12 GROUP BY m.modalidad_contratacion
13 ORDER BY duracion_promedio_dias DESC
14 """).show(truncate=False)
```

[Stage 89:=====>

(3 + 5) / 8]

modalidad_contratacion	cantidad_contratos	duracion_promedio_dias	promedio_dias_adicionado
Licitación Pública Acuerdo Marco de Precios	1	2248.0	304.0
Concurso de méritos abierto	38	543.3	85.18
Licitación pública Obra Publica	31	391.8	85.03
Selección Abreviada Menor Cuantía Sin Manifestación Interés	6	291.2	10.33
Licitación pública	34	285.0	24.38
Contratación Directa (con ofertas)	241	280.1	10.09
Contratación régimen especial (con ofertas)	154	235.1	20.44
Contratación directa	13982	186.1	7.23
Selección Abreviada de Menor Cuantía	188	179.2	11.81
Contratación régimen especial	2434	171.6	13.58
Selección abreviada subasta inversa	167	150.4	8.8
Mínima cuantía	1048	117.5	4.96
Enajenación de bienes con subasta	1	34.0	15.0
Enajenación de bienes con sobre cerrado	1	30.0	0.0

9. Estado de los contratos por trimestre de firma

```
In [19]: 1 spark.sql("""
2 SELECT t.anio, t.trimestre, es.estado_contrato, COUNT(*) AS cantidad_contratos
3 FROM {DB_NAME}.hecho_contratos h
4 JOIN {DB_NAME}.dim_tiempo t      ON h.sk_tiempo_firma = t.sk_tiempo
5 JOIN {DB_NAME}.dim_estado_contrato es ON h.sk_estado = es.sk_estado
6 GROUP BY t.anio, t.trimestre, es.estado_contrato
7 ORDER BY t.anio, t.trimestre
8 """).show(30, truncate=False)
```

[Stage 94:=====> (178 + 10) / 200]

anio	trimestre	estado_contrato	cantidad_contratos
2016	1	Modificado	1
2016	3	Aprobado	1
2016	3	Cerrado	3
2016	4	Aprobado	1
2017	1	Cerrado	7
2017	1	cedido	1
2017	1	Aprobado	6
2017	1	Prorrogado	1
2017	2	terminado	5
2017	2	Aprobado	3
2017	2	Prorrogado	1
2017	3	terminado	2
2017	3	Aprobado	5
2017	3	Cerrado	5
2017	3	Modificado	1
2017	4	Aprobado	17
2017	4	Cerrado	15
2017	4	Modificado	6
2017	4	terminado	4
2018	1	Aprobado	46
2018	1	cedido	3
2018	1	Cerrado	50
2018	1	Modificado	43
2018	1	terminado	24
2018	2	Modificado	14
2018	2	terminado	9
2018	2	Cerrado	11
2018	2	Aprobado	18
2018	3	Aprobado	41
2018	3	terminado	12

only showing top 30 rows

10. Categorías de proceso con mayor valor promedio por contrato

In [21]:

```
1 spark.sql("""
2 SELECT c.codigo_categoria_principal, c.descripcion_proceso, COUNT(*) AS cantidad_contratos, ROUND(AVG(h.valor_contrato), 2)
3 FROM {DB_NAME}.hecho_contratos h
4 JOIN {DB_NAME}.dim_categoria c ON h.sk_categoria = c.sk_categoria
5 GROUP BY c.codigo_categoria_principal, c.descripcion_proceso
6 ORDER BY valor_promedio DESC
7 LIMIT 15
8 """).show(truncate=False)
```

[Stage 97:=====] (148 + 8) / 200

	codigo_categoria_principal	descripcion_proceso	cantidad_contratos	valor_promedio
VI.72152100		CONSTRUCCIÓN DE LA TERCERA ETAPA DEL AUDITORIO MUNICIPAL DE TOCANCIPÁ	1	3.7911256553E10
VI.92121800		Contratar el servicio de arrendamiento de vehículos blindados para ser utilizados como medidas de protección de la población objeto del programa de prevención y protección de la unidad nacional de protección a nivel nacional; de conformidad con las condiciones y especificaciones técnicas establecidas	1	3.4852747314E10
VI.93121607		Contribuir a la implementación del componente restaurativo de los proyectos que hacen parte de las sanciones; TOAR y medidas de contribución que cuentan con orden judicial de la JEP.	2	3.09474355605E10
VI.72121200		RECONSTRUCCIÓN DEL CENTRO DE FORMACIÓN TURÍSTICA; GENTE DE MAR Y DE SERVICIOS SUBSEDE DEL SENA EN LA ISLA DE PROVIDENCIA; INCLUIDA EN EL PLAN DE ACCIÓN ESPECÍFICO; DEPARTAMENTO ARCHIPIÉLAGO DE SAN ANDRÉS PROVIDENCIA Y SANTA CATALINA- VERSION N°2 EN EL MARCO DEL CONTRATO INTERADMINISTRATIVO N°220005	1	1.021612022305E10
VI.81112002		IMPLEMENTACIÓN DE SOLUCIONES BASADAS EN LA NATURALEZA (SBN) PARA LA RESTAURACIÓN SOCIOECOLÓGICA; ADAPTACIÓN AL CAMBIO CLIMÁTICO Y PROMOCIÓN DE LA SOBERANÍA ALIMENTARIA EN ZONAS RURALES DE BOSQUE SECO TROPICAL EN EL DEPARTAMENTO DE CORDOBA	2	9.3890441875E9
VI.72121400		Aunar esfuerzos técnicos; administrativos y financieros; entre el Departamento Administrativo para la Prosperidad Social - Fondo de Inversión para la Paz - PROSPERIDAD SOCIAL - FIP y la ENTIDAD TERRITORIAL; para la ejecución de obras de infraestructura social vial; con el propósito de aportar a la	5	9.19923738553E9
VI.78111500		HORAS DE VUELO PARA EL SERVICIO DE TRANSPORTE DE PASAJEROS Y CARGA DEL EJÉRCITO NACIONAL; EN AERONAVES ALA FIJA	1	8.97064294E9
VI.73152104		PRESTAR EL SERVICIO TÉCNICO DE MANTENIMIENTO Y SUMINISTRO DE REPUESTOS A LOS EQUIPOS MARCA KOSME Y KRONES DE LA LÍNEA DE ENVASADO N°1 Y N°3 Y LOS EQUIPOS DE LA LÍNEA NO. 4 DE LA FÁBRICA DE LICORES Y ALCOHOLES DE ANTIOQUIA -EICE	1	8.540428302E9
VI.81102200		INTERVENTORIA TÉCNICA; ADMINISTRATIVA; LEGAL; FINANCIERA; SOCIAL; AMBIENTAL Y DE SEGURIDAD Y SALUD EN EL TRABAJO PARA LAS OBRAS PENDIENTES POR EJECUTAR DURANTE LA ETAPA DE CONSTRUCCIÓN DEL CONTRATO DE OBRA IDU-136-2007. ADemás; RECIBO FINAL DE LAS OBRAS; BALANCE FINANCIERO; TÉCNICO; Y DOCUMENTAL DEL	4	8.5153674095E9
VI.72111000		Aunar esfuerzos técnicos; administrativos; sociales y financieros con la Junta de Acción Comunal de Barrio El Cagua Segundo Sector para el mejoramiento y/o adecuación de vivienda en el marco del programa obras con capital social en el municipio de Soacha; con el fin de contribuir al desarrollo so	4	8.34679405475E9
VI.25102000		ADQUISICIÓN DE KIT DE BLINDAJE PARA VEHÍCULOS TÁCTICOS HMMV PERTENECIENTES AL PROGRAMA DE SEGURIDAD EN CARRETERAS NACIONALES DEL INVÍAS	1	7.546138663E9
VI.50192700		CONTRATAR EL SERVICIO DE PRODUCCIÓN; SUMINISTRO Y DISPENSACIÓN DE DIETAS HOSPITALARIAS Y ALIMENTACIÓN PARA MÉDICOS INTERNOS Y RESIDENTES DE LA SUBRED INTEGRADA DE SERVICIOS DE SALUD CENTRO ORIENTE E.S.E	4	6.9975215335E9
VI.72141100		REALIZAR EL MANTENIMIENTO DE CANALES; CERRAMIENTO Y ROCERÍA DEL AEROPUERTO LOS COLONIZADORES DE SARAVENA; ARAUCA	10	6.6845281763E9
VI.72121406		MEJORAMIENTO; MANTENIMIENTO Y ADECUACIÓN DE LA INFRAESTRUCTURA EDUCATIVA DE LAS SEDES PRIMARIA; CORINTO; CURISI Y PRINCIPAL PERTENECIENTES A LA INSTITUCIÓN EDUCATIVA TÉCNICA AGROPECUARIA DEL MUNICIPIO DE PAJARITO; DEPARTAMENTO DE BOYACÁ; PARA ATENDER LA EMERGENCIA DE CALAMIDAD PÚBLICA DE ACUERDO A D	3	6.66291608283E9
VI.83101807		Mejorar la calidad y confiabilidad del servicio de energía eléctrica en los barrios subnormales del Mercado de Comercialización del OPERADOR DE RED ubicados en los municipios del Sistema Interconectado Nacional - SIN- conforme los reglamentos técnicos vigentes; legalizando los usuarios y adecuando l	1	5.899481457E9

Conclusiones

- Este notebook explota el cubo `secop_dw` con 10 consultas analíticas, incluyendo el uso correcto de `dim_tiempo` como dimensión de rol en múltiples combinaciones (firma, inicio+fin simultáneamente).
- Todas combinan `hecho_contratos` con una o más dimensiones, demostrando el valor analítico del modelo.
- Con esto se cierra el flujo completo: Verificación de entorno → Extracción → Cubo → Transformación → Cargue → Explotación analítica.